



北京航空航天大学
BEIHANG UNIVERSITY



操作系统 内存管理基础

廖晓坚

liaoqxj@buaa.edu.cn



内容提要

- 存储管理基础
- 页式内存管理
- 段式内存管理
- 虚拟存储管理
- 存储管理实例



■ 存储管理基础

- 存储器管理目标
- 存储器硬件发展
- 存储器管理的功能
- 程序的装载与链接
- 存储器分配方法

■ 页式内存管理

■ 段式内存管理

■ 虚拟存储管理

■ 存储管理实例



存储管理基础

- 存储器管理目标
- 存储器硬件
- 存储器管理的功能
- 程序的链接与装载
- 存储器分配方法



存储管理基础

- **存储器管理目标**
- 存储器硬件
- 存储器管理的功能
- 程序的链接与装载
- 存储器分配方法



存储器管理的目标

■ 研究对象：以内存为中心的存储资源

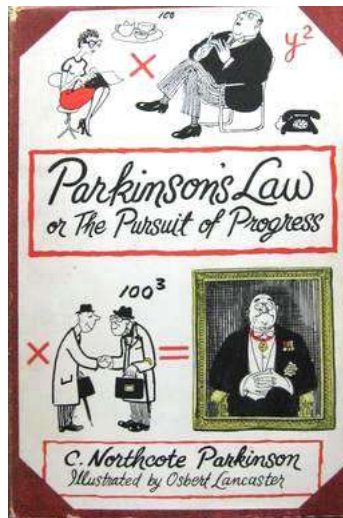
- 程序在内存中运行

■ 用户角度（程序员）

- 容量大
- 速度快（性能）
- 独立拥有，不受干扰（安全）

■ 资源管理角度

- 为多用户提供服务
- 效率、利用率、能耗



帕金森定律(Parkinson)

work expands so as to fill the time available for its completion



无论存储器空间有多大，程序都能将其耗尽



存储管理的基本需求

基本需求:

■ 从每个计算机使用者（程序员）的角度:

- 1. 整个空间都归我使用;
- 2. 不希望任何第三方因素妨碍我的程序的正常运行

■ 从计算机平台提供者的角度:

- 尽可能同时为多个用户提供服务

分析:

■ 计算机至少同时存在两个程序: 一个用户程序和一个服务程序（操作系统）

- 每个程序具有的地址空间应该是相互独立的
- 每个程序使用的空间应该得到保护



存储管理基础

- 存储器管理目标
- **存储器硬件**
- 存储器管理的功能
- 程序的链接与装载
- 存储器分配方法



存储器硬件

■ RAM: Random Access Memory

- SRAM
- DRAM
- SDRAM、DDR SDRAM

■ ROM: Read Only Memory

- PROM、EPROM、EEPROM等

■ Flash memory

- NOR
- NAND

■ Disk

■ Tape

易失性

非易失性

外存



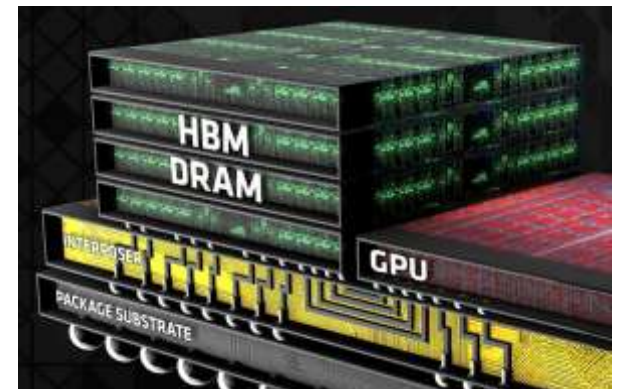
存储器硬件

■ 存储器的功能：为CPU保存指令和数据

- 操作系统代码和数据
- 应用程序代码和数据

■ 存储器的发展方向：高速、大容量和小体积。

- 内存在访问速度方面的发展：DDR、GDDR、LPDDR、HBM等





存储器硬件

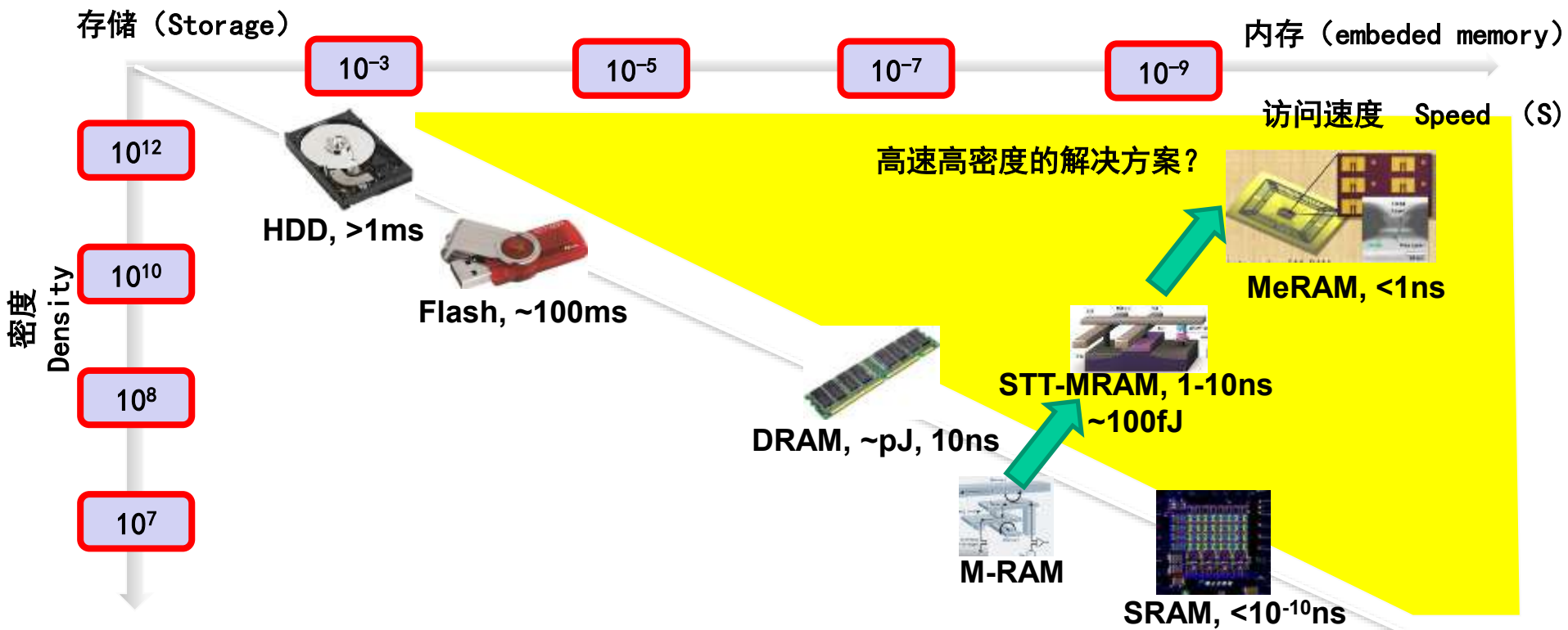
- **静态存储器（SRAM）：**读写速度快，生产成本低，多用于容量较小的高速缓冲存储器
- **动态存储器（DRAM）：**读写速度较慢，集成度高，生产成本低，多用于容量较大的主存储器

	SRAM	DRAM
存储信息方式	触发器（RS）	电容
破坏性读出	否	是
定期刷新	不需要	需要
送地址方式	行列同时送	行列分两次送
运行速度	快	慢
发热量	大	小
存储成本	高	低
集成度	低	高



蕴含巨大的挑战

- 非易失性存储器件 (NVM) 的发展是否有突破的机会?
- SSD (Flash)、PCM、忆阻器、自旋电子器件



K.L.Wang, Spin orbit interaction engineering of magnetic memory for energy efficient electronics systems. **NVMTS 2015**

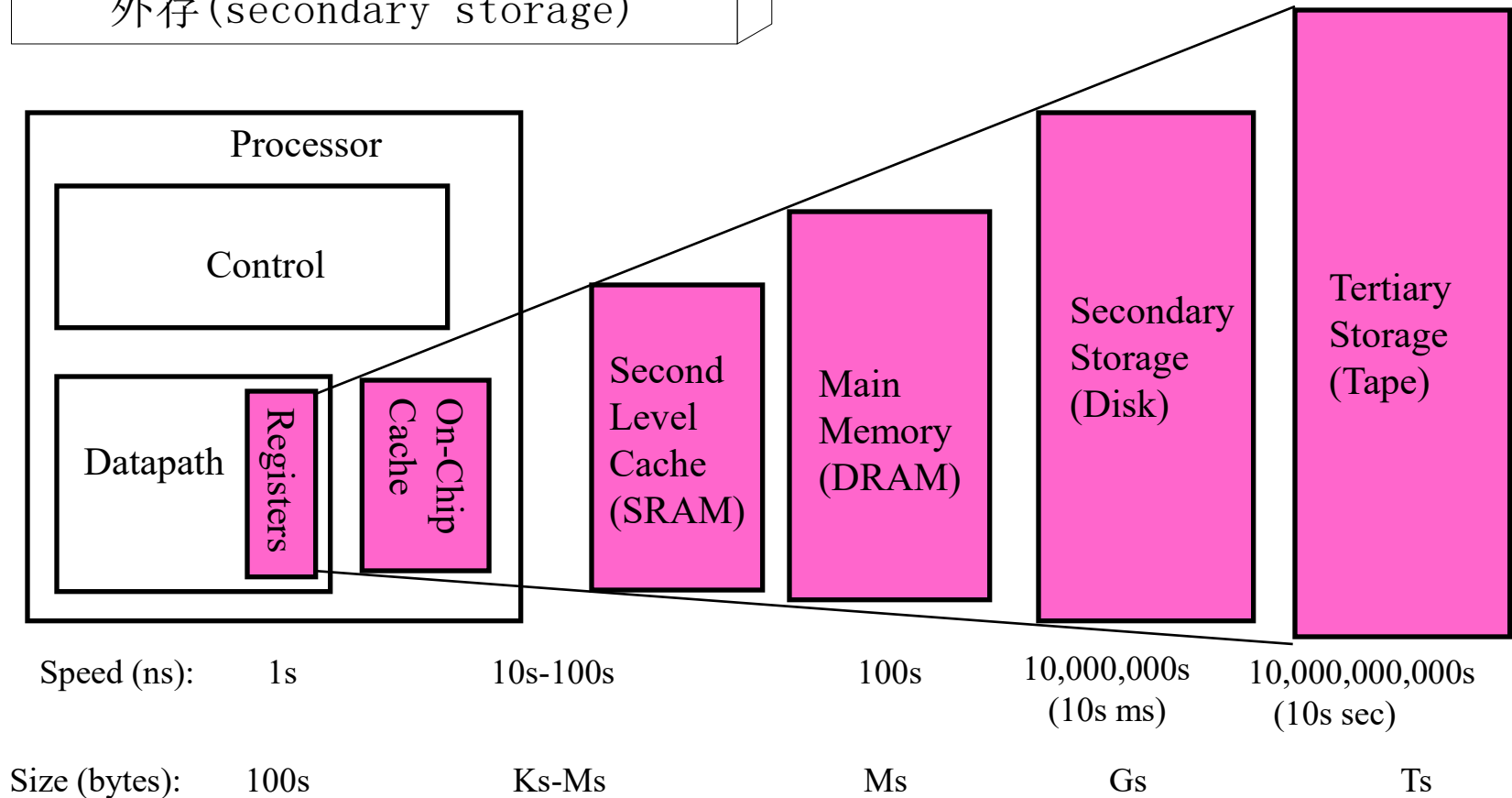


存储组织

- 存储组织：在存储技术和CPU寻址技术许可的范围内组织合理的存储结构。其依据是访问速度匹配关系、容量要求和价格
- 例如：“寄存器-内存-外存”结构和“寄存器-缓存-内存-外存”结构
- 典型的层次式存储组织：访问速度越来越慢，容量越来越大，价格越来越便宜
- 最佳状态应是各层次的存储器都处于均衡的繁忙状态（如：缓存命中率正好使主存读写保持繁忙）



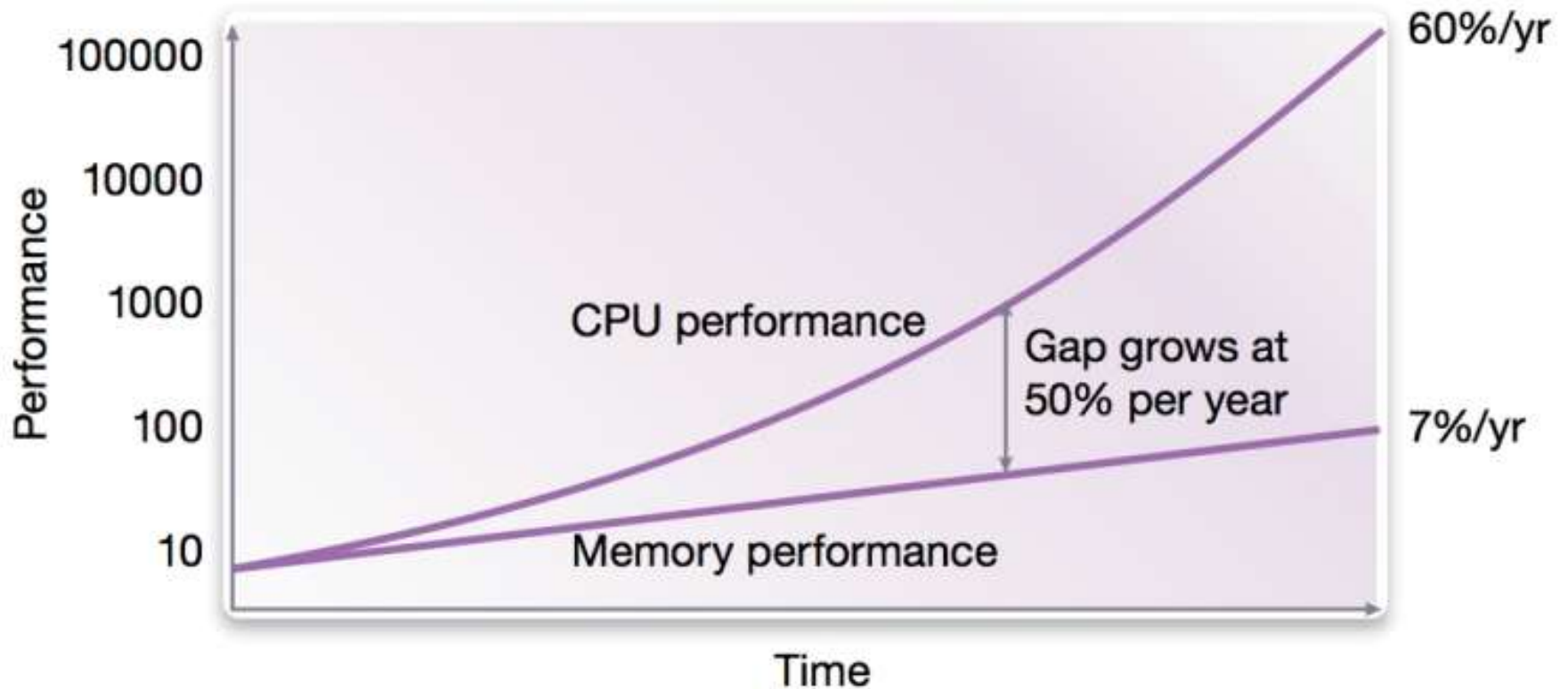
存储层次结构





Memory Wall

Moore's law effect





存储管理基础

- 存储器管理目标
- 存储器硬件
- **存储器管理的功能**
- 程序的链接与装载
- 存储器分配方法



存储管理的功能

- **地址变换：**将物理内存进行抽象，使得每个进程认为分配给它的私有且连续的内存地址空间
- **存储分配和回收：**为进程分配或回收内存空间，提升存储器利用率
- **存储共享和保护：**代码和数据共享，对地址空间的访问权限
- **存储器扩充：**它涉及存储器的逻辑组织和物理组织
 - 由应用程序控制：覆盖；
 - 由OS控制：交换（整个进程空间），请求调入和预调入（部分进程空间）



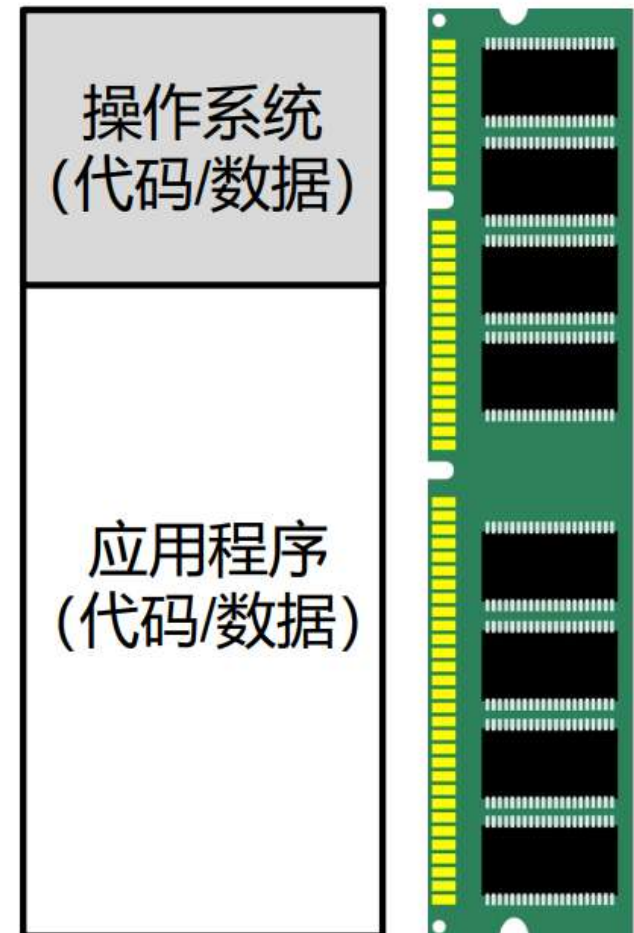
早期的计算机系统

■ 硬件

- 物理内存容量小

■ 软件

- 单个应用程序 + （简单）操作系统
- 直接面对物理内存编程
- 各自使用物理内存的一部分





多道程序时代

■ 多用户多程序

- 计算机很昂贵，多人同时使用（远程连接）

■ 分时复用CPU资源

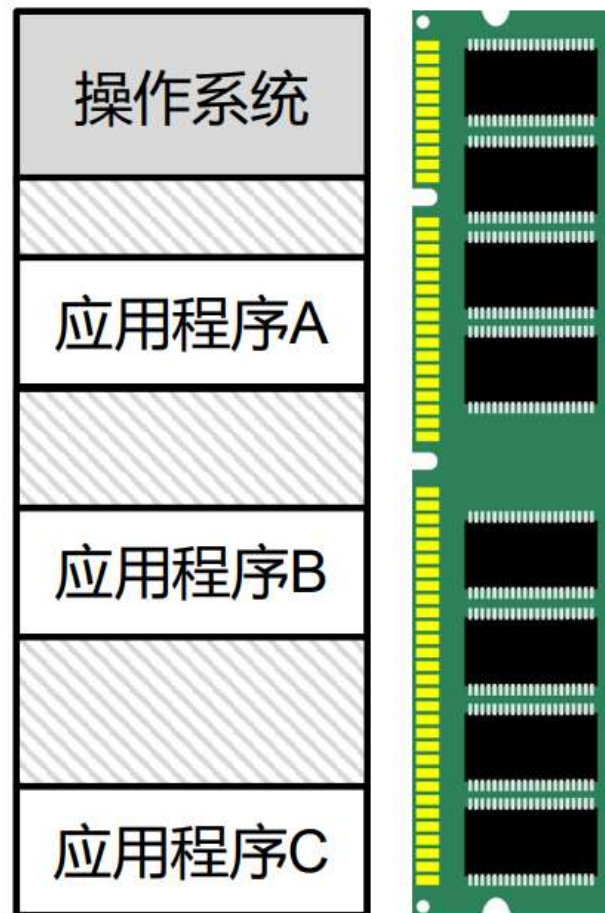
- 保存恢复寄存器速度很快

■ 分时复用物理内存资源

- 将全部内存写入磁盘开销太高

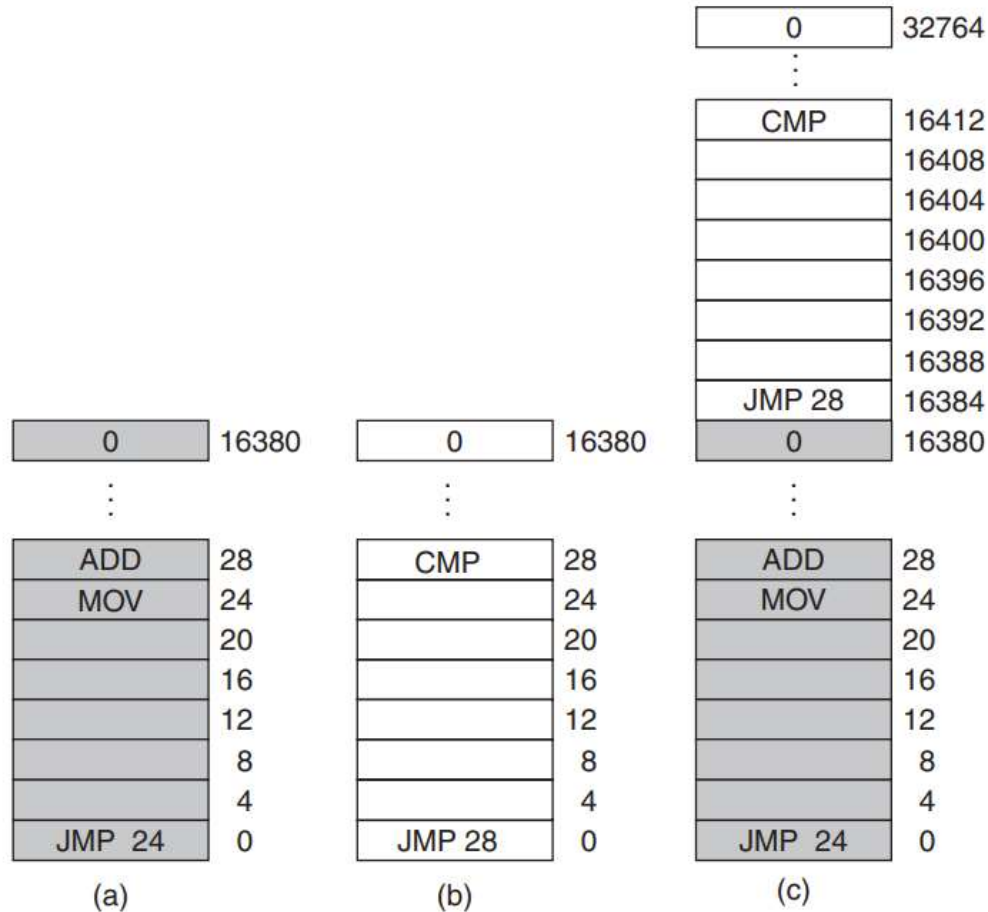
■ 同时使用、各占一部分物理内存

- 没有安全性（隔离性）





多道程序共享内存



(a) A 16-KB program. (b) Another 16-KB program. (c) The two programs loaded consecutively into memory



使用物理地址的缺点

■ 物理地址对应用是可知的，导致：

- 一个应用会因其他应用的加载而受到影响
- 一个应用可通过自身的内存地址，猜测出其他应用的加载位置

■ 是否可以让应用看不见物理地址？

- 不用关心其他进程，不受其他进程的影响
- 看不见其他进程的信息，更强的隔离能力

如何让OS与不同的应用程序都高效又安全地使用物理内存资源？

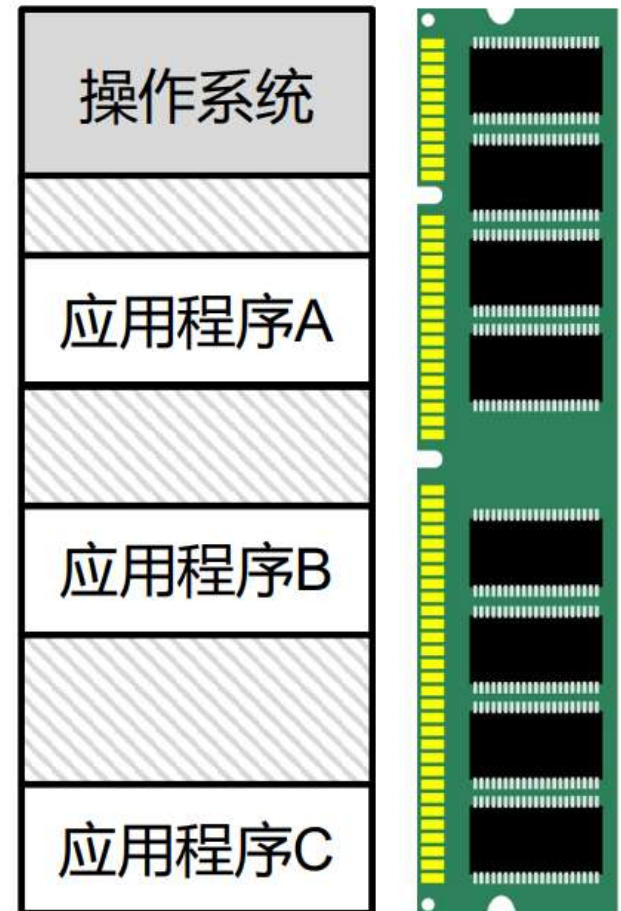


程序运行的地址空间

- 进程中的地址不是最终的物理地址
- 在进程运行前无法计算出物理地址
- 因为：不能确定进程被加载到内存什么地方

地址重定位的支持！

地址转换、地址变换、地址翻译、地址映射
Translation、Mapping





地址重定位

■ 逻辑地址（相对地址，虚拟地址）

- 用户程序经过编译、汇编后形成目标代码，目标代码通常采用相对地址的形式，其首地址为0，其余地址都相对于首地址而编址
- 不能用逻辑地址在内存中读取信息

■ 物理地址（绝对地址，实地址）

- 内存中存储单元的地址可直接寻址

为了保证CPU执行指令时可正确访问内存单元，需要将用户程序中的逻辑地址转换为运行时可由机器直接寻址的物理地址，这一过程称为地址重定位



静态重定位与动态重定位

■ 静态重定位

- 当用户程序加载到内存时，一次性实现逻辑地址到物理地址的转换
- 一般可以由软件完成

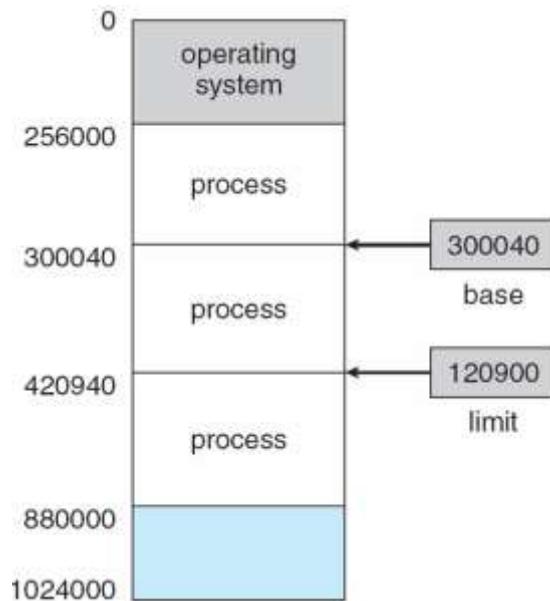
■ 动态重定位

- 在进程执行过程中进行地址变换
- 即逐条指令执行时完成地址转换
- 需要硬件部件支持



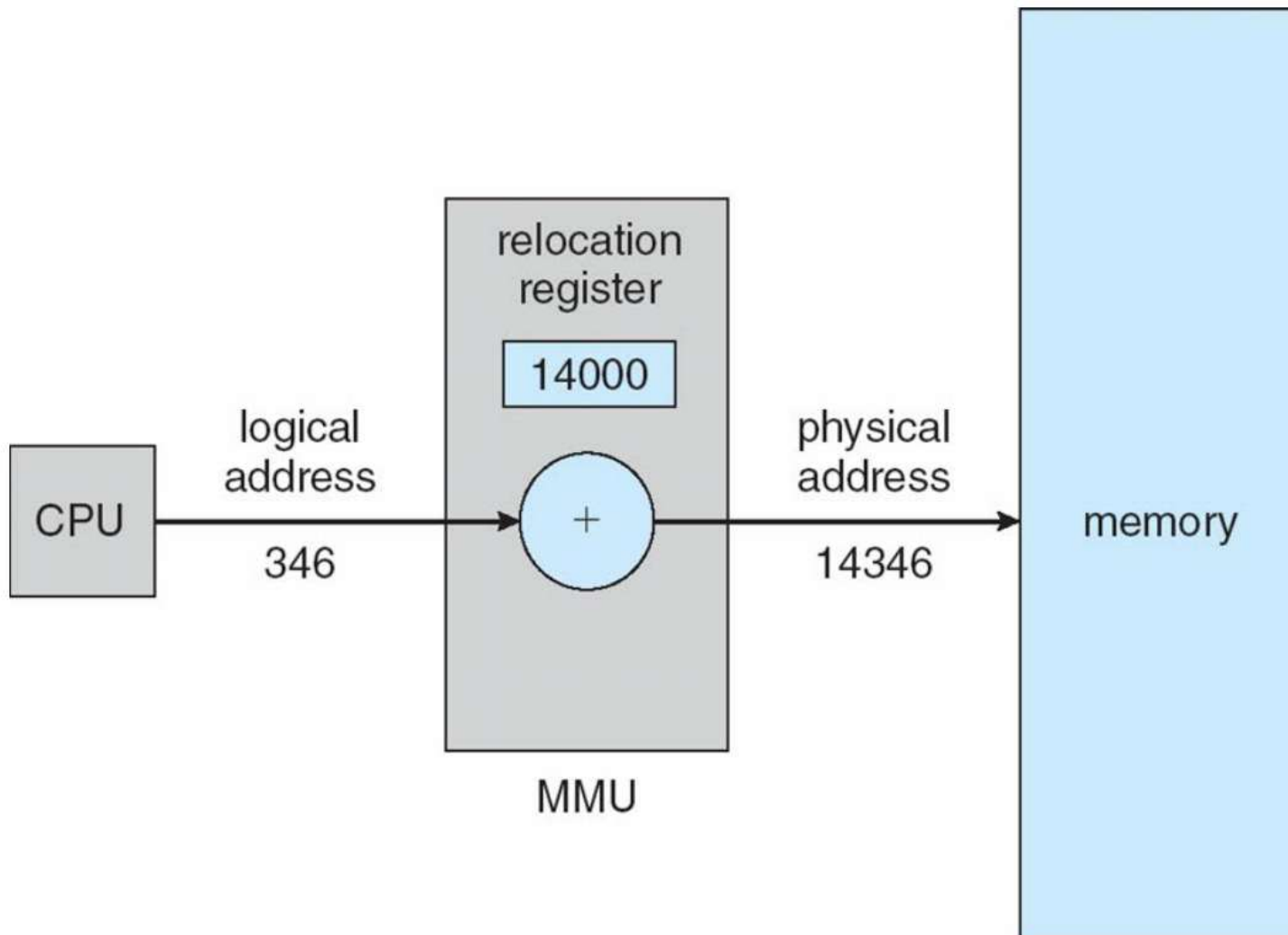
基于硬件的动态重定位

- CPU两个硬件寄存器，基址（base）和限制（limit）寄存器
- 进程使用内存引用都是虚拟地址，硬件将虚拟地址加上基址寄存器，得到物理地址
- 每次CPU发出访存，都会检查地址是否在base和limit之间



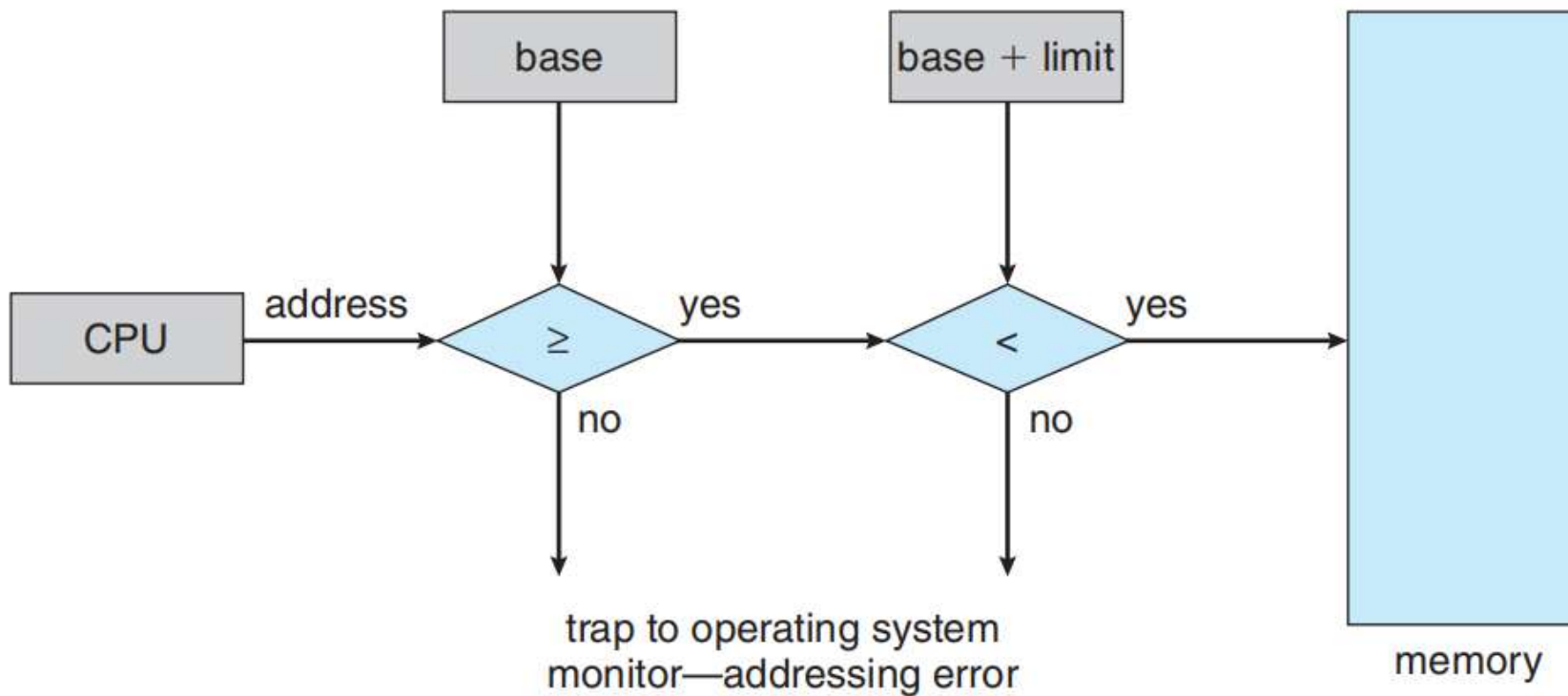


动态重定位的实现



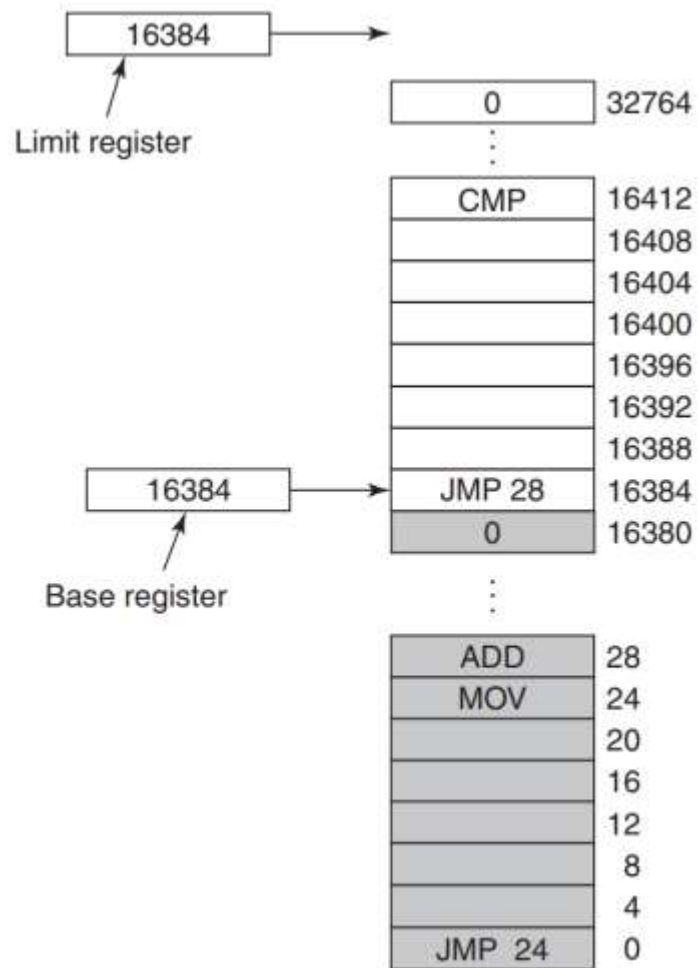


硬件单元实现地址保护





多道程序共享内存



(c)

Base and limit registers can be used to give each process a separate address space



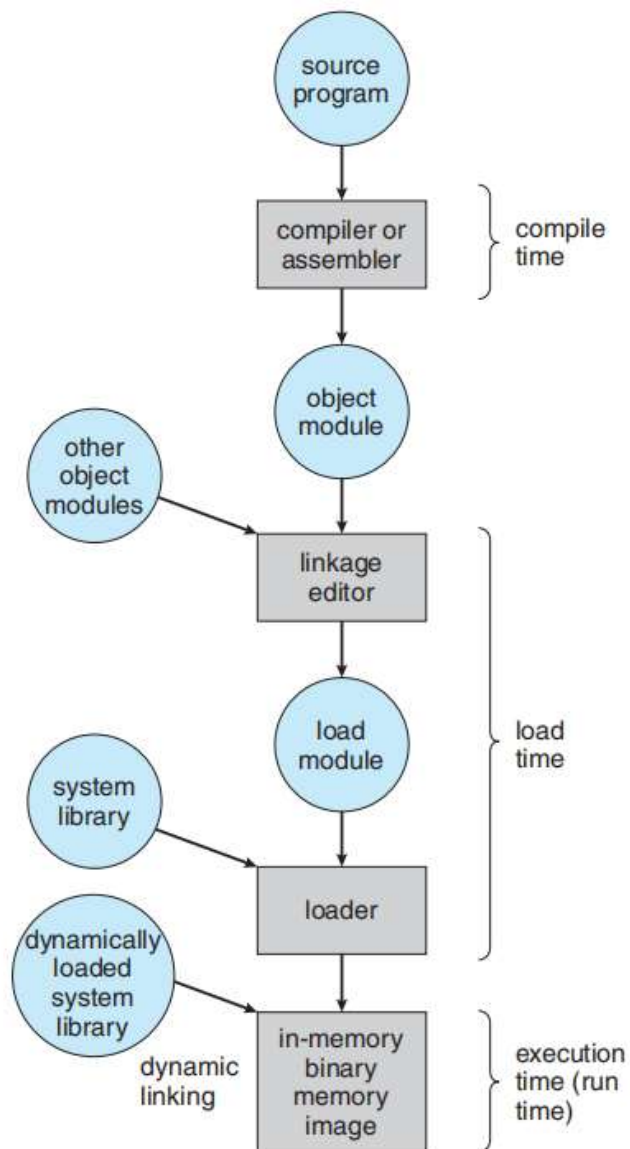
存储管理基础

- 存储器管理目标
- 存储器硬件
- 存储器管理的功能
- 程序的链接与装载
- 存储器分配方法



程序的链接与装载

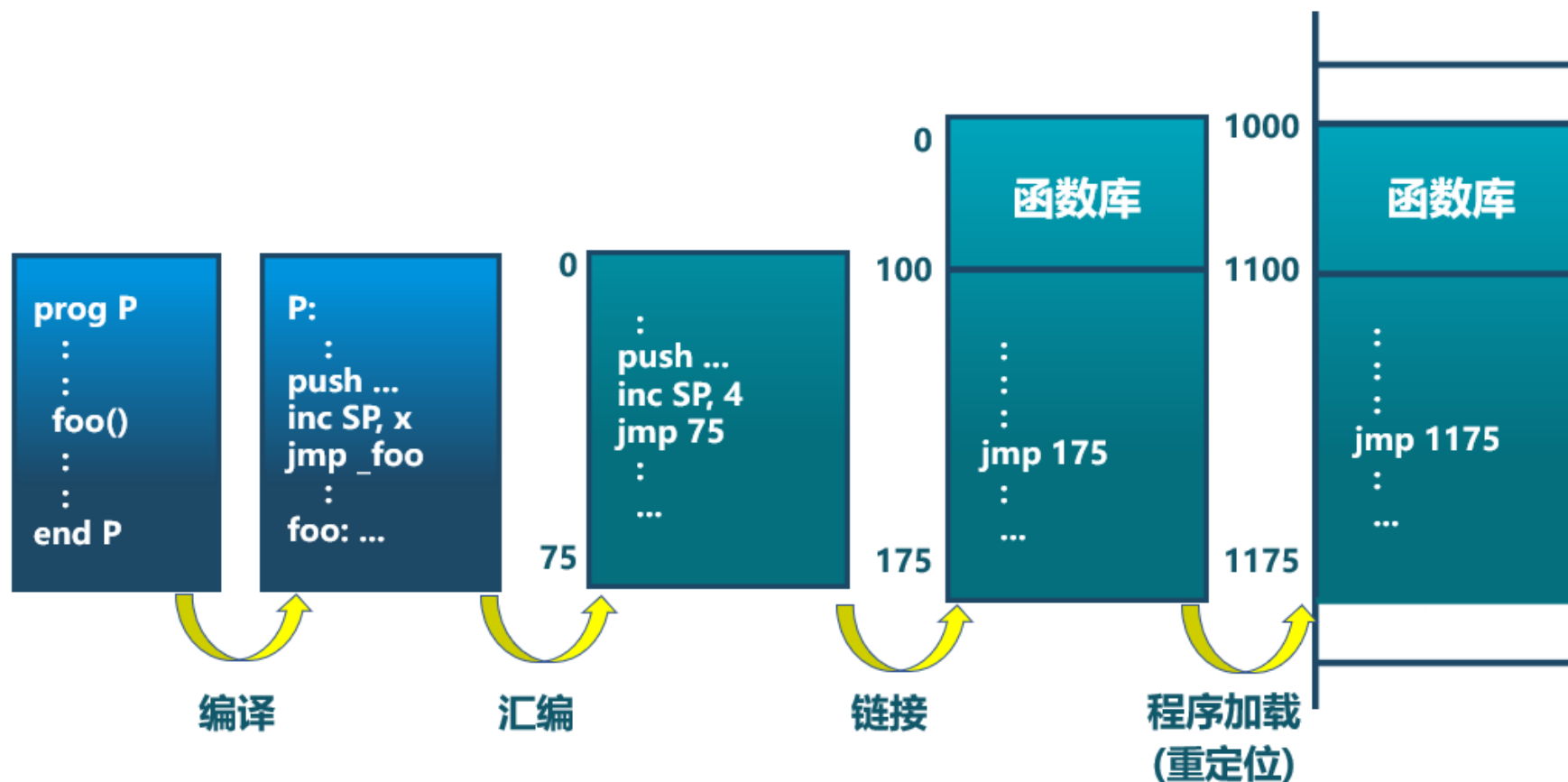
- 编译
- 链接
- 装载
- 执行





程序的链接与装载

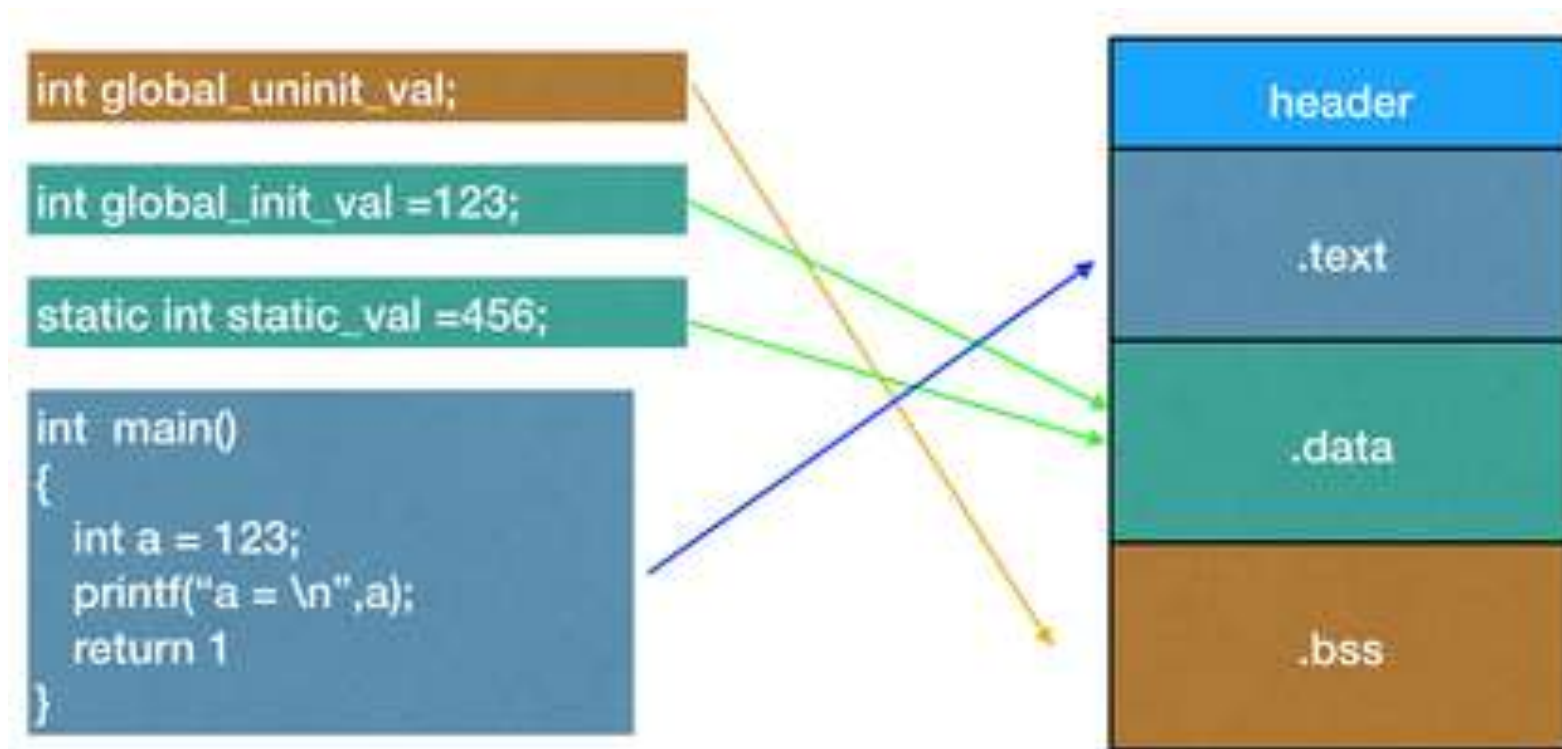
逻辑地址生成





程序加载基础知识

- 一个程序本质上都是由bss段、data段、text段三个组成的。在C语言之类的程序编译完成之后，已初始化的全局变量保存在data 段中，未初始化的全局变量保存在bss段中。





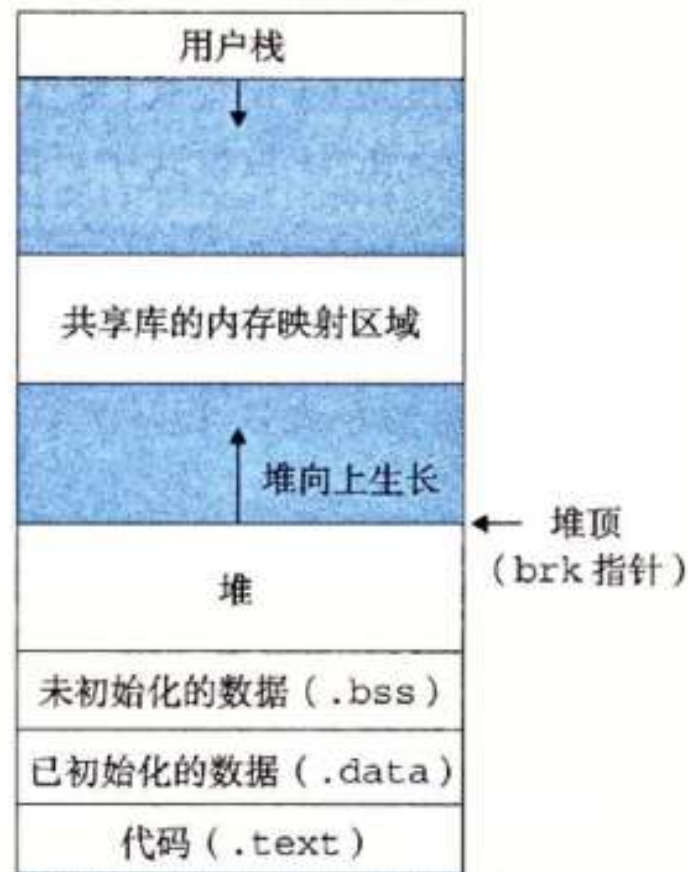
程序加载基础知识

- **bss段**: (bss segment) 用来存放程序中**未初始化的全局变量**的一块内存区域。bss是英文Block Started by Symbol的简称。bss段属于静态内存分配。
- **data段**: **数据段** (data segment) 用来存放程序中**已初始化的全局变量**的一块内存区域。数据段属于静态内存分配。
- **text段**: **代码段** (code segment/text segment) 用来存放**程序执行代码**的一块内存区域。这部分区域的大小在程序运行前就已经确定, 并且内存区域通常属于只读(某些架构也允许代码段为可写, 即允许修改程序)。在代码段中, 也有可能包含一些只读的常数变量, 例如字符串常量等。



程序加载基础知识

- text和data段都在可执行文件中，由系统从可执行文件中加载，而bss段不在可执行文件中，由系统初始化
- 一个装入内存的可执行程序，除了bss、data和text段外，还需构建一个栈（stack）和一个堆（heap）





程序加载基础知识

■ 栈(stack): 存放、交换临时数据的内存区

- 用户存放程序局部变量的内存区域，（但不包括static声明的变量，static意味着在数据段中存放变量）
- 保存/恢复调用现场。在函数被调用时，其参数也会被压入发起调用的进程栈中，并且待到调用结束后，函数的返回值也会被存放回栈中

■ 堆 (heap) : 存放进程运行中动态分配的内存段

- malloc()函数分配内存，free()函数释放内存
- 它的大小并不固定，可动态扩张或缩减。当进程调用malloc等函数分配内存时，新分配的内存就被动态添加到堆上（堆被扩张）；当利用free等函数释放内存时，被释放的内存从堆中被剔除（堆被缩减）



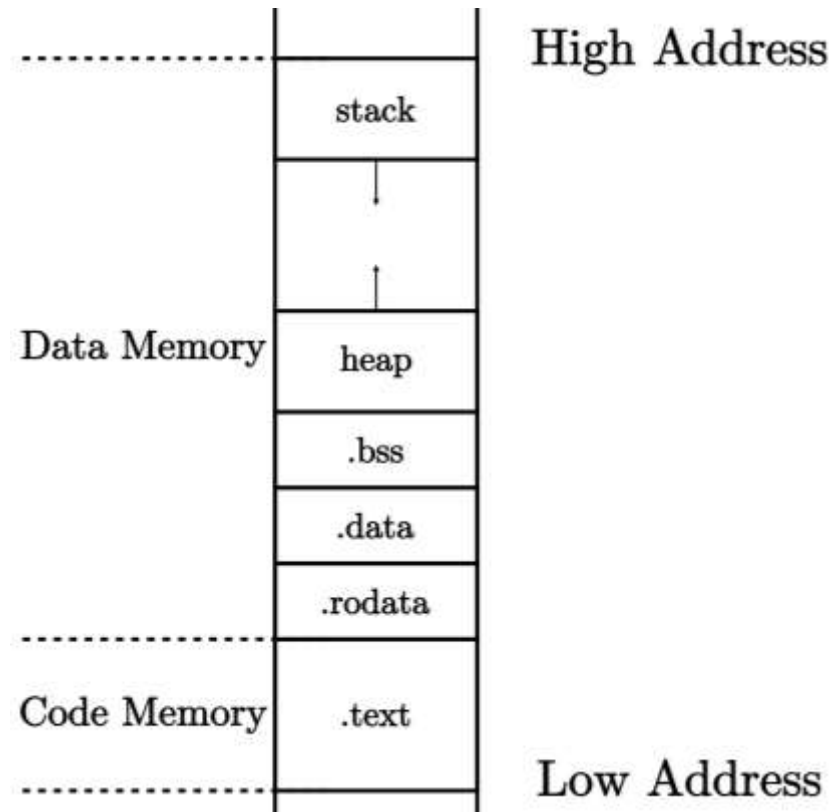
堆和栈的内存分配

■ 分配方式：动态内存分配

- 栈由编译器管理：隐式分配
- 堆的分配和释放由程序员管理：显式分配

■ 分配大小

- 栈是由高地址向低地址生长的数据结构，是一块连续的内存，能从栈中获得的内存较小，编译期间确定大小
- 堆是由低地址向高地址生长的数据结构，是一个不连续的储存空间，内存获取比较灵活，也较大





ELF目标文件格式

■ 目标文件：指编译器生成的文件

- 目标文件一般也叫作 ABI（Application Binary Interface，应用程序二进制接口），目标文件和目标平台是二进制兼容的

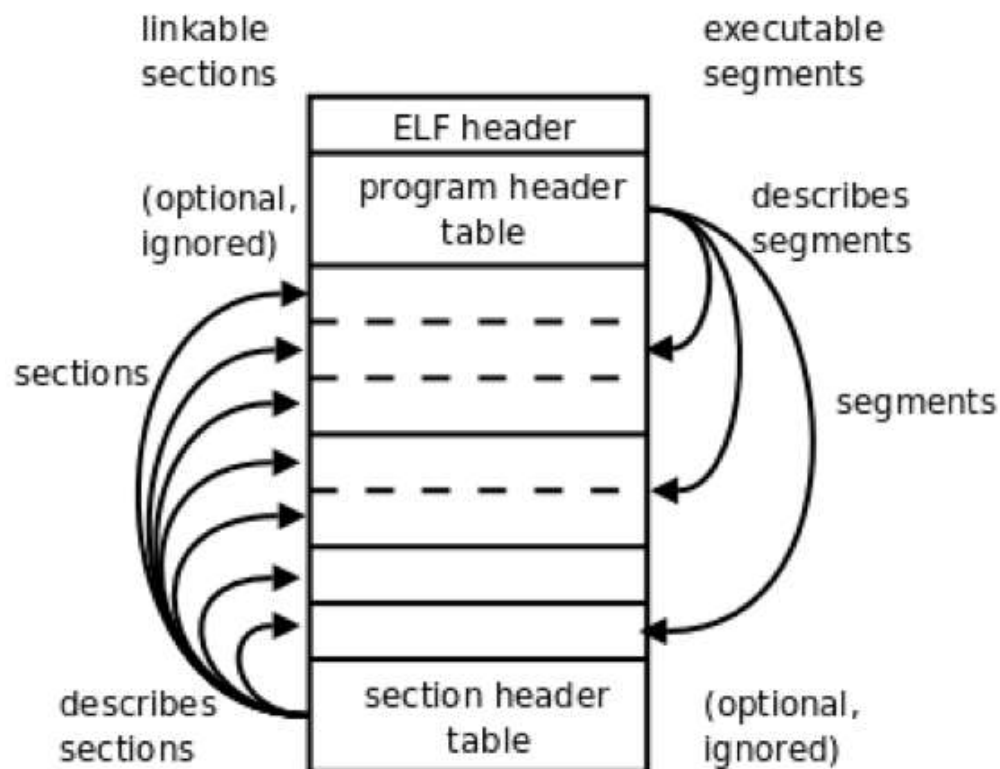
■ ELF（Executable and Linkable Format）即可执行的和可链接的格式，是一个目标文件格式的标准。ELF 格式的文件用于存储 Linux 程序

- 最古老的目标文件格式是 a.out，后来发展成 COFF 格式，现在常用的格式有 PE（Windows）和 ELF（Linux）

■ ELF 是一种对象文件的格式，用于定义不同类型的对象文件中都有什么内容、以什么样的格式放这些内容



ELF目标文件格式



ELF头

程序头表（可省略）

.text

.rodata

.data

.....

节头表



ELF目标文件格式

■ Section（节）：链接视图

- ELF文件的链接视图的基本单位，由编译器生成，用于描述代码、数据、符号表等不同功能的数据块
- 用途：服务于链接器，帮助其完成符号解析、重定位等操作。例如，链接器需要将多个目标文件中的.text节合并成最终的代码段

Section名称	描述
.text	可执行代码（函数指令）
.data	已初始化的全局/静态变量
.rodata	只读数据（如字符串常量）
.bss	未初始化的全局/静态变量（预留空间）
.symtab	符号表（函数/变量名称与地址映射）
.rela.text	重定位信息（修正代码地址引用）



ELF目标文件格式

■ Segment（段）：执行视图

- ELF文件的执行视图的基本单位，由操作系统加载器处理，描述程序运行时的内存布局
- 主要在执行阶段使用，操作系统根据segment信息将程序加载到内存中
- 将多个section按内存属性合并，优化内存占用（如将只读的.text和.rodata合并到一个只读段）

Segment类型	描述
LOAD	需要加载到内存的段（代码/数据）
DYNAMIC	动态链接信息（如依赖的共享库）
INTERP	程序解释器路径（如/lib/ld-linux.so）
GNU_STACK	栈权限（是否允许可执行代码）



Section与Segment的关系

- Segment包含多个Section:
- 链接器将多个Section合并为一个Segment, 优化内存布局
- 示例:
 - 代码段 (LOAD类型): 合并.text、.rodata等只读Section
 - 数据段 (LOAD类型): 合并.data、.bss等可写Section

特性	Section (节)	Segment (段)
作用阶段	编译/链接阶段	程序加载/执行阶段
核心目标	逻辑分组代码和数据	物理内存映射和权限控制
数量	多 (几十个)	少 (通常2-5个)
查看工具	readelf -S / objdump -h	readelf -l / objdump -p



ELF Section字段解析

bss	此节存放用于程序内存映象的未初始化数据。此节类型是SHT_NOBITS。
.comment	此节存放版本控制信息。
.data和.data1	此节存放用于程序内存映象的初始化数据。
.debug	此节存放符号调试信息。
.dynamic	此节存放动态连接信息。
.dynstr	此节存放动态连接所需的字符串，代表的是与符号表项有关的名字。
.dynsym	此节存放的是“符号表”中描述的动态连接符号表。
.fini	此节存放与进程中指代码有关的执行指令。
.got	此节存放全程偏移量表。
.hash	此节存放一个符号散列表。
.init	此节存放组成进程初始化代码的执行指令。
.interp	此节存放一个程序解释程序的路径名。
.line	此节存放符号调试中使用的行号信息，描述源程序与机器指令间对应关系
.note	此节存放供其他程序检测兼容性，一致性的特殊信息。
.plt	此节存放过程连接表。
.relname/.relaname	此节存放重定位信息。
.rodata/.rodata1	此节存放进程映象中不可写段的只读数据。
.shstrtab	此节存放节名。
.strtab	此节存放的字符串标识与符号表项有关的名字。
.symtab	此节存放符号表。
.text	此节存放正文，也称程序的执行指令。



ELF文件头的定义

ELF头描述文件组成

```
typedef struct {  
    unsigned char    e_ident[16];        /* 标志本文件为目标文件，提供  
                                           机器无关的数据，可实现对文  
                                           件内容的译码与解释*/  
  
    unsigned char    e_type[2];          /* 标识目标文件类型 */  
    unsigned char    e_machine[2];      /* 指定必需的体系结构 */  
    unsigned char    e_version[4];      /* 标识目标文件版本 */  
    unsigned char    e_entry[4];        /* 指向起始虚地址的指针 */  
    unsigned char    e_phoff[4];        /* 程序头表的文件偏移量 */  
    unsigned char    e_shoff[4];        /* 节头表的文件偏移量 */  
    unsigned char    e_flags[4];        /* 针对具体处理器的标志 */  
    unsigned char    e_ehsize[2];       /* ELF 头的大小 */  
    unsigned char    e_phentsize[2];   /* 程序头表每项的大小 */  
    unsigned char    e_phnum[2];       /* 程序头表项的个数 */  
    unsigned char    e_shentsize[2];   /* 节头表每项的大小 */  
    unsigned char    e_shnum[2];       /* 节头表项的个数 */  
    unsigned char    e_shstrndx[2];    /* 与节名字符串表相关的节头表  
                                           项的索引 */  
}  
Elf32_Ehdr;
```



ELF文件头的定义

e_ident

这一部分是文件的标志，用于表明该文件是一个ELF文件。ELF文件的头四个字节为magic number。

e_type

用于标明该文件的类型，如可执行文件、动态链接库、可重定位文件等。

e_machine

表明体系结构，如x86，x86_64，MIPS，PowerPC等等。

e_version

文件版本

e_entry

程序入口的虚拟地址

e_phoff

程序头表在该ELF文件中的位置(具体地说是偏移)。ELF文件可以没有程序头表。



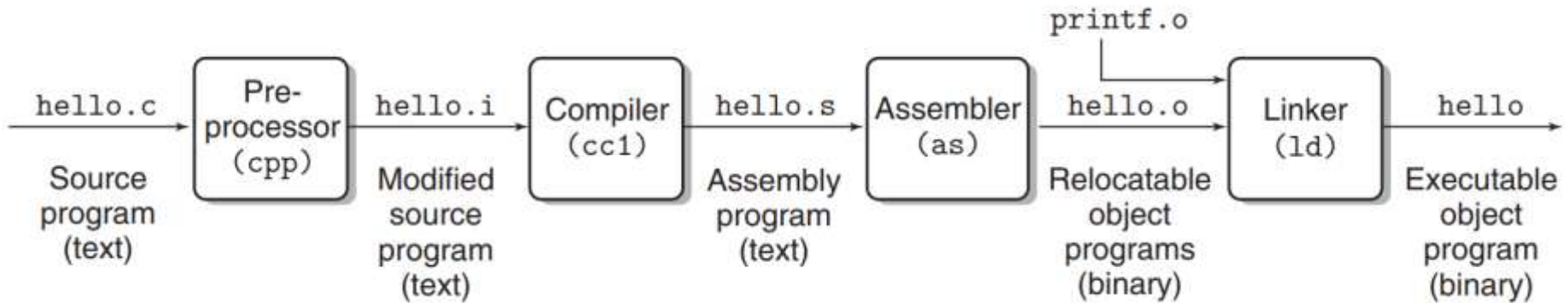
ELF文件头的定义

e_shoff	节头表的位置。
e_elflags	针对具体处理器的标志。
e_ehsize	ELF 头的大小。
e_phentsize	程序头表每项的大小。
e_phnum	程序头表项的个数。
e_shentsize	节头表每项的大小。
e_shnum	节头表项的个数。
e_shstrndx	与节名字符串表相关的节头表。



程序编译

■ 程序从源代码到可执行文件的步骤：预处理、编译、汇编、链接





预处理

■ 预处理时编译器完成的具体工作如下

- 删除所有的注释 “//”和 “/**/”
- 删除所有的 “#define”，展开所有的宏定义
- 处理所有的条件预编译指令
- 处理 “#include”预编译指令，将被包含的文件插入该预编译指令的位置，这一过程是递归进行的
- 添加行号和文件名标识

■ 如下指令将对 `hello.c` 进行预处理

■ `gcc -E hello.c -o hello.i`



预处理结果

```
720 |
721 extern char *ctermid (char *__s) __attribute__ ((__nothrow__ , __leaf__))
722 | __attribute__ ((__access__ (__write_only__, 1)));
723 # 867 "/usr/include/stdio.h" 3 4
724 extern void flockfile (FILE *__stream) __attribute__ ((__nothrow__ , __leaf__));
725
726
727
728 extern int ftrylockfile (FILE *__stream) __attribute__ ((__nothrow__ , __leaf__));
729
730
731 extern void funlockfile (FILE *__stream) __attribute__ ((__nothrow__ , __leaf__));
732 # 885 "/usr/include/stdio.h" 3 4
733 extern int __uflow (FILE *);
734 extern int __overflow (FILE *, int);
735 # 902 "/usr/include/stdio.h" 3 4
736
737 # 2 "hello.c" 2
738
739 # 2 "hello.c"
740 ~ int main() {
741     printf("hello, world\n");
742     return 0;
743 }
744
```




编译

- 编译时，gcc首先要检查代码的规范性、是否有语法错误等，以确定代码实际要做的工作。在检查无误后，gcc把代码翻译成汇编语言

- `gcc -S hello.i -o hello.s`

-S: 该选项只进行编译而不进行汇编，即仅生成汇编代码，不进一步翻译为机器指令

```
9  main:
10  v .LFB0:
11      .cfi_startproc
12      endbr64
13      pushq   %rbp
14      .cfi_def_cfa_offset 16
15      .cfi_offset 6, -16
16      movq    %rsp, %rbp
17      .cfi_def_cfa_register 6
18      leaq    .LC0(%rip), %rax
19      movq    %rax, %rdi
20      call    puts@PLT
21      movl    $0, %eax
22      popq    %rbp
23      .cfi_def_cfa 7, 8
24      ret
25      .cfi_endproc
```



汇编

■ 汇编生成ELF格式目标文件，主要节区

- ELF 格式文件。程序编译后生成的目标文件至少 含有 3 个节区 (Section)，分别为 .text、.data 和 .bss

■ as hello.s -o hello.o

```
hello.o:      file format elf64-x86-64
```

```
Disassembly of section .text:
```

```
0000000000000000 <_start>:
```

```
0:  b8 01 00 00 00      mov     eax,0x1
5:  bf 01 00 00 00      mov     edi,0x1
a:  48 be 00 00 00 00 00  movabs  rsi,0x0 ; <-- 注意这里，msg的地址还是0x0
11:  00 00 00
```

后续重定位



链接

■ 编译C程序的时候，是以.c文件作为编译单元的

- 编译：.c→.o
- 编译时函数定义在不同文件，无法知道地址。
 - E重定位目标文件
 - U未解析符号
 - D已定义符号

■ 文件合并：

- 将这些.o文件链接到一起，形成最终的可执行文件
- 在链接时，链接器会扫描各个目标文件，将之前未填写的地址填写上，从而生成一个真正可执行的文件

■ 重定位(Relocation)

- 将之前未填写的地址填写的过程



链接过程的合并

- 链接是将各种代码和数据部分收集起来并组合成为一个单一文件的过程，这个文件可被加载（或复制）到内存中并执行

`gcc -o hello hello.o`

```
369 0000000000001050 <puts@plt>:
370 | 1050: f3 0f 1e fa          endbr64
371 | 1054: f2 ff 25 75 2f 00 00    bnd jmp *0x2f75(%rip)      # 3fd0 <puts@GLIBC_2.2.5>
372 | 105b: 0f 1f 44 00 00          nopl 0x0(%rax,%rax,1)

447 0000000000001149 <main>:
448 | 1149: f3 0f 1e fa          endbr64
449 | 114d: 55                   push %rbp
450 | 114e: 48 89 e5             mov %rsp,%rbp
451 | 1151: 48 8d 05 ac 0e 00 00  lea 0xeac(%rip),%rax
452 | 1158: 48 89 c7             mov %rax,%rdi
453 | 115b: e8 f0 fe ff ff . . . . . call 1050 <puts@plt>
454 | 1160: b8 00 00 00 00      mov $0x0,%eax
455 | 1165: 5d                   pop %rbp
456 | 1166: c3                   ret
```

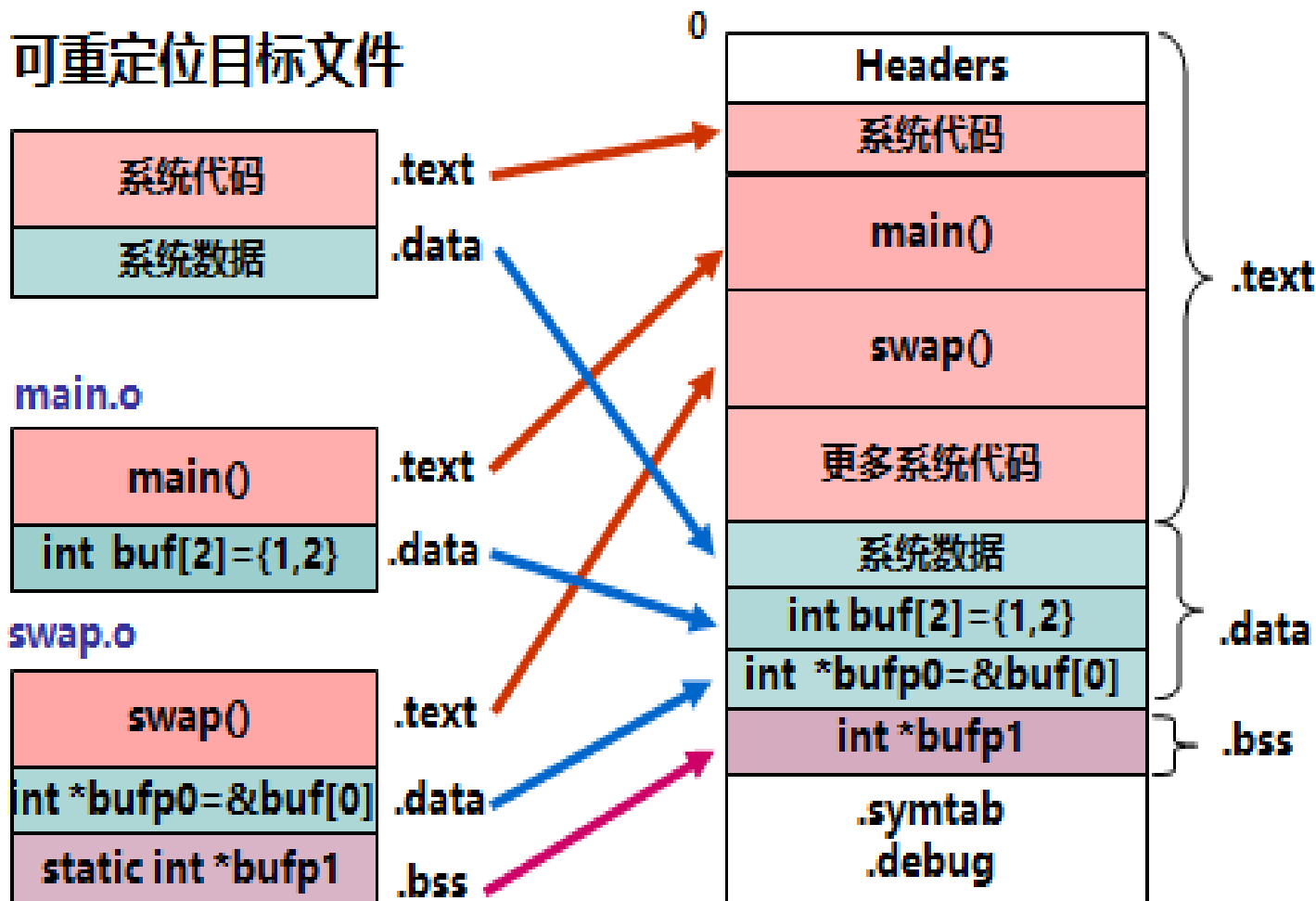


链接过程的合并

链接本质：合并相同的“节”

可执行目标文件

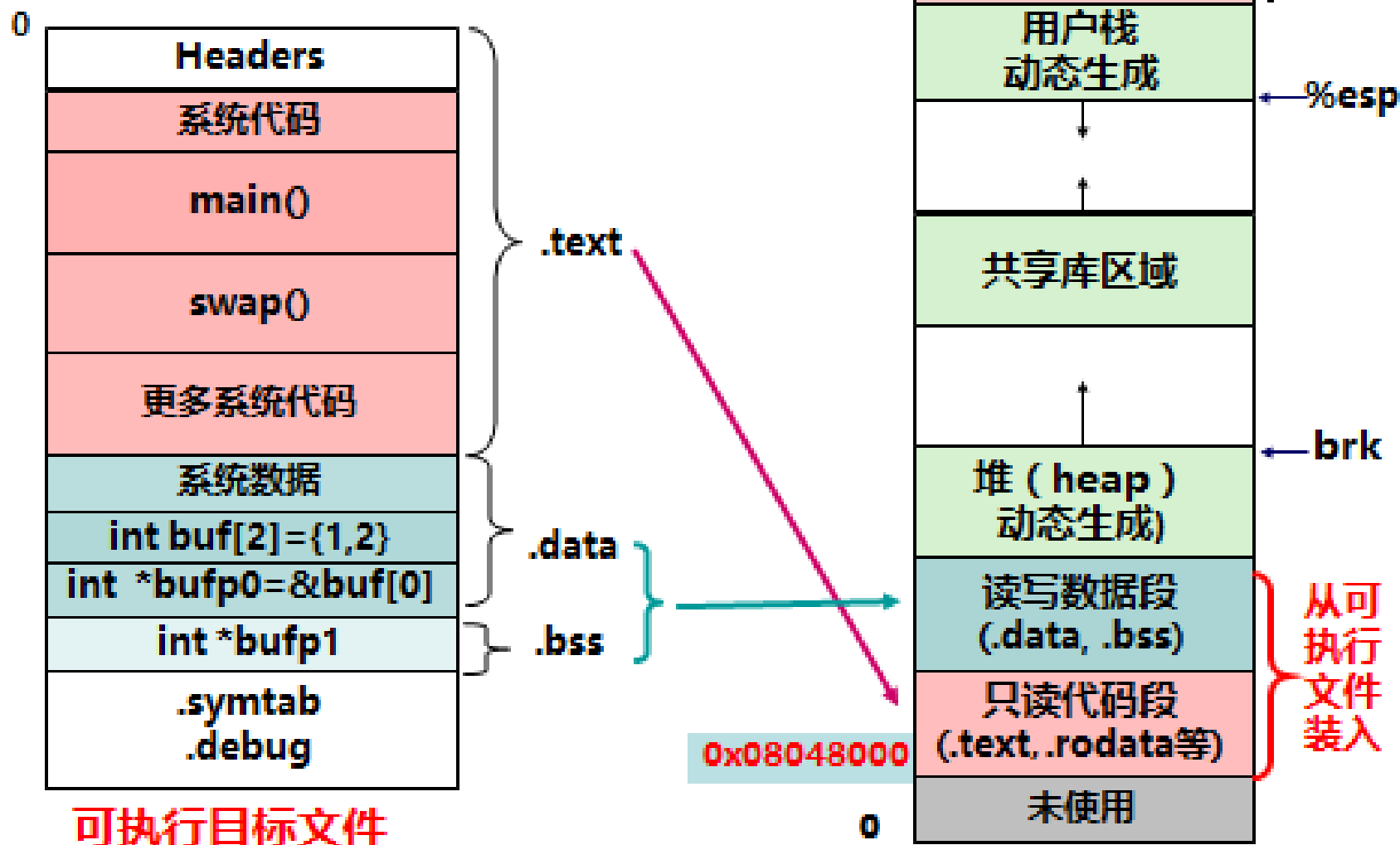
可重定位目标文件





可执行文件的内存映像(x86)

程序(段)头表描述如何映射！





链接与库

- 在可执行文件的生成过程中，最为复杂的部分就是链接
- 链接从过程上讲分为**符号解析**和**重定位**两部分
- 根据链接时机的不同，又分为**静态链接**和**动态链接**两种



符号与符号解析

- **符号包含全局变量和全局函数。例如printf就是一个符号，程序需要在函数库中找到这个符号**
- **链接器上下文中的3种不同符号如下**
 - **全局符号：**由模块定义并能被其他模块引用的全局符号。全局链接器符号对应非静态的C函数以及被定义为不带C static属性的全局变量
 - **外部符号：**由其他模块定义并被模块引用的全局符号。这些符号称为外部符号（external），对应定义在其他模块中的C函数和变量
 - **本地符号：**只被模块定义和引用的本地符号。有的本地链接器符号对应带static属性的C函数和全局变量



符号与符号解析

```
/* 1. 本地符号 (Local Symbol) */
static int local_static_var = 10;          // static 全局变量: 只能在本文件内访问
static void local_static_func(void) {      // static 函数: 只能在本文件内调用
    return;
}

/* 2. 全局符号 (Global Symbol) */
int global_var = 100;                      // 非 static 全局变量: 可被其他文件引用

/* 3. 外部符号 (External Symbol) */
extern int external_var;                   // 外部变量: 定义在其他文件中
extern void external_func(void);          // 外部函数: 定义在其他文件中

/* 另一个全局函数, 它使用了上述三种符号 */
void main_func(void) {
    int a = local_static_var;              // 引用 本地符号
    int b = global_var;                    // 引用 全局符号
    int c = external_var;                  // 引用 外部符号

    local_static_func();                   // 调用 本地符号
    external_func();                       // 调用 外部符号
}
```



静态链接

■ 在程序装载到内存和运行前，其所有目标模块及所需要的库函数链接和装配成一个完整的可执行程序且此后不再拆分

- 链接与装载过程相对独立，链接和装载程序可独立设计
- 但不支持内存空间中目标模块的单副本、不利于模块共享

■ 优点

- 代码的装载速度快，执行速度也比较快，对外部环境依赖度低。

■ 缺点

- 编译时它会把需要的所有代码都链接进去，应用程序相对比较大。
如果多个应用程序使用同一库函数，会被装载多次，浪费内存



动态链接 —— 装载时链接

■ 在程序装入内存前，不进行程序各目标模块的链接，而是在程序装载时，一边装载一边链接，生成一个可执行程序

- 装载目标模块时，若发生外部模块调用，将引发相应外部目标模块的搜索、装载和链接

■ 优点

- 简单，开发方便。各目标模块相对独立存在，便于个别目标模块的修改或更新，且不影响程序的装载和执行
- 若发现所需某目标模块已在内存，可直接进行链接且无须再次装载，支持目标模块的共享

■ 缺点

- 无论程序中某个函数是否真的被执行，只要它在依赖库中，程序启动时都会被加载和链接，这会导致启动速度变慢。



动态链接 —— 运行时链接

■ 将某些目标模块或库函数的链接推迟到执行时才进行

- 若发现被调用模块或库函数尚未链接，先在内存中搜索以查看其是否装入内存
 - 若已装入，则直接将其链接到调用者程序中
 - 否则在外存上的搜索，并装入内存和进行链接，生成可执行程序

■ 优点

- 避免程序执行过程中不被调用的某些目标模块在执行前进行链接和装载而引起的开销，提高系统资源利用率和系统效率

■ 缺点

- 代码稍微复杂，需要手动处理指针和错误检查



装载时链接 vs. 运行时链接

维度	装载时链接	运行时链接
触发者	操作系统/动态链接器	程序员/程序代码
发生时间	进程启动时（main执行前）	进程运行期间（main执行后）
依赖声明	写在可执行文件的动态节中（DT_NEEDED）	代码中通过dlopen 显式指定
符号解析	启动时全部解析完成	调用 dlsym 时才解析
典型用途	标准C库、基础依赖库	插件、驱动、功能模块



程序的装载

装载前的工作：

- shell调用fork()系统调用，创建一个子进程。

装载工作：

- 子进程调用execve()加载program(即要执行的程序)。

程序如何被加载：

- 加载器在加载程序的时候只需要看ELF文件中和segment相关的信息即可。我们用readelf工具将segment读取出来：读出的信息分为两部分，一部分是各segment的具体信息，另一部分是section和segment之间的对应关系。
- 其中Type为Load的segment是需要被加载到内存中的部分。



程序的装载流程

■ 读取ELF头部的魔数(Magic Number)，以确认该文件确实是ELF文件。

- ELF文件的头四个字节依次为'0x7f'、'E'、'L'、'F'。
- 加载器会首先对比这四个字节，若不一致，则报错。

■ 找到段表项（执行视图）

- ELF头部会给出的段表起始位置在文件中的偏移，段表项的大小，以及段表包含了多少项。根据这些信息可以找到每一个段表项。

■ 对于每个段表项解析出各个段应当被加载的虚地址，在文件中的偏移。以及在内存中的大小和在文件中的大小。（段在文件中的大小小于等于内存中的大小）。



程序的装载流程

- 对于每一个段，根据其在内存中的大小，为其分配足够的物理页，并映射到指定的虚地址上，再将文件中的内容拷贝到内存中。
- 若ELF中记录的段在内存中的大小大于在文件中的大小，则多出来的部分用0进行填充。
- 动态链接库处理：依赖解析、按序加载、**符号解析和重定位**
- 设置进程控制块中的PC为ELF文件中记载的入口地址。
- 控制权交给进程开始执行！



程序的装载流程

细节:

- 一个segment在文件中的大小是小于等于其在内存中的大小。
- 如果在文件中的大小小于在内存中大小，那么在载入内存时通过**补零**使其达到其在内存中应有的大小

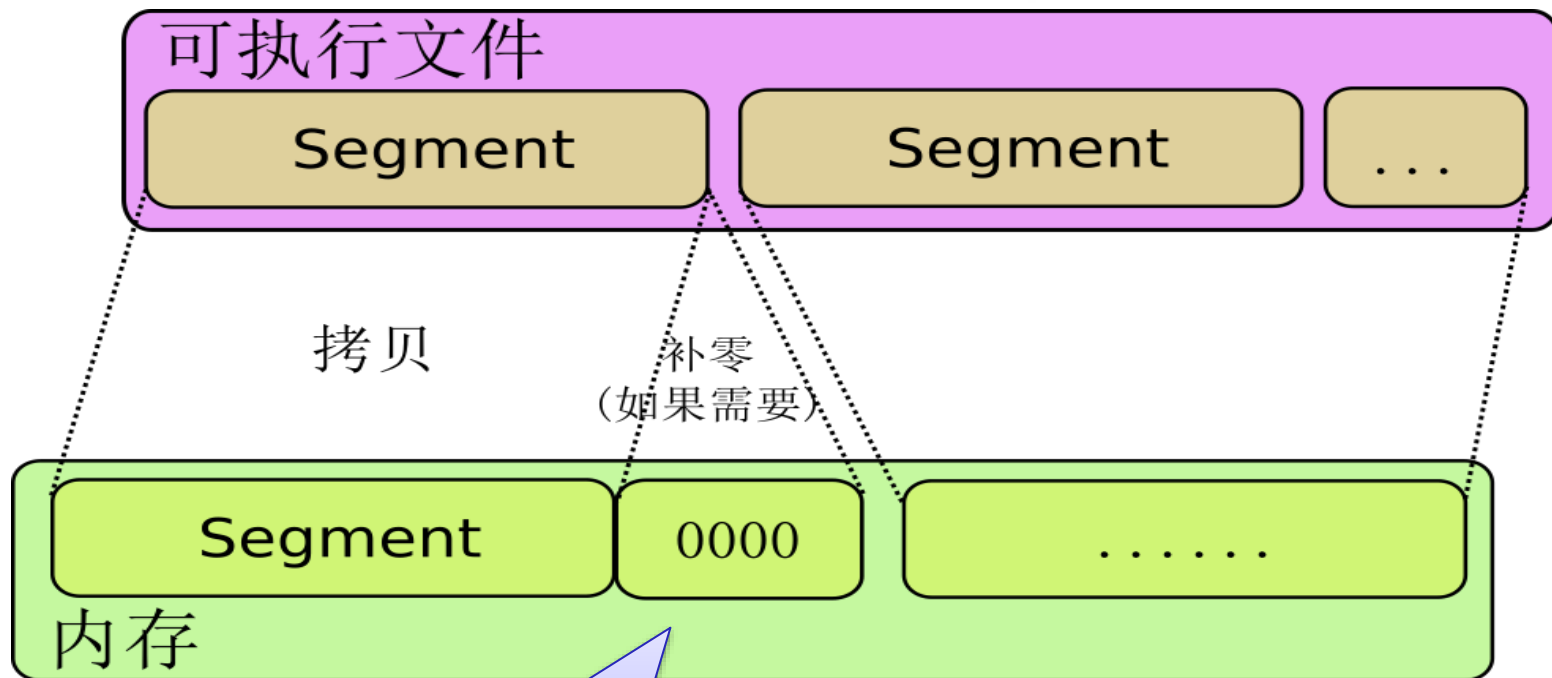
```
#include <stdio.h>

int data_var = 0x1234;    // 已初始化, 去 .data
int bss_var[1000];        // 未初始化, 去 .bss

int main() {
    printf("%d\n", bss_var[0]); // 理论上输出0
    return 0;
}
```

程序的装载流程

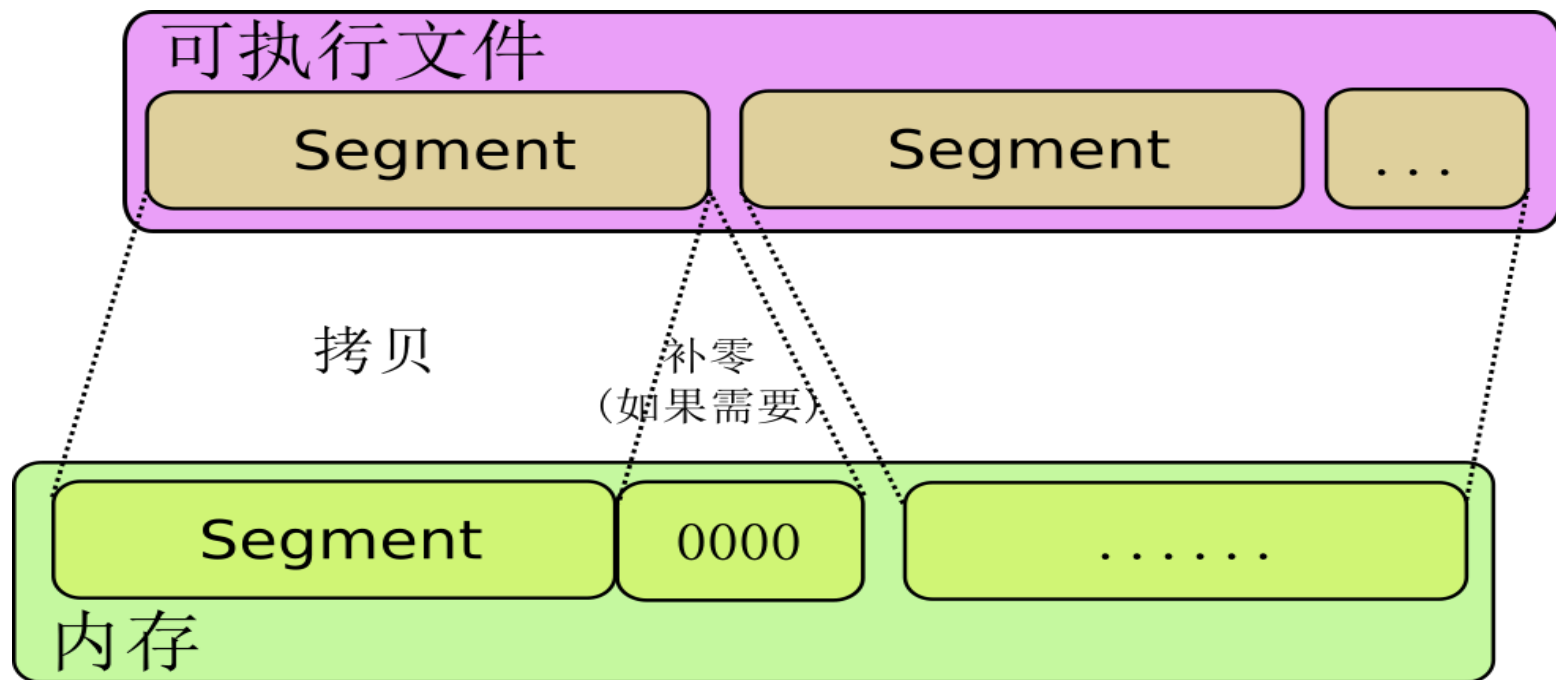
代码段和数据段都在segment中



文件大小小于内存大小，补0



程序的运行



各个segment都装入内存后，控制权交给应用，
从应用入口开始执行



存储管理基础

- 存储器管理目标
- 存储器硬件
- 存储器管理的功能
- 程序的链接与装载
- 存储器分配方法



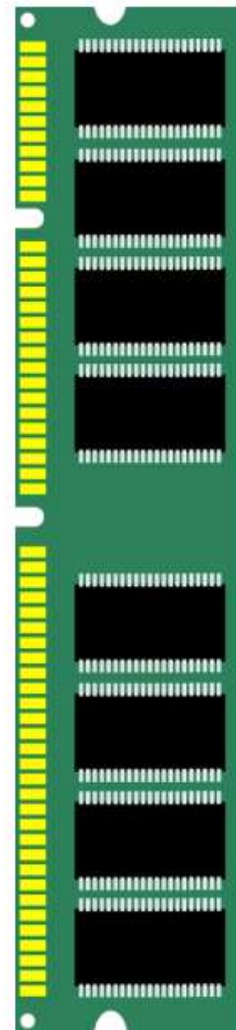
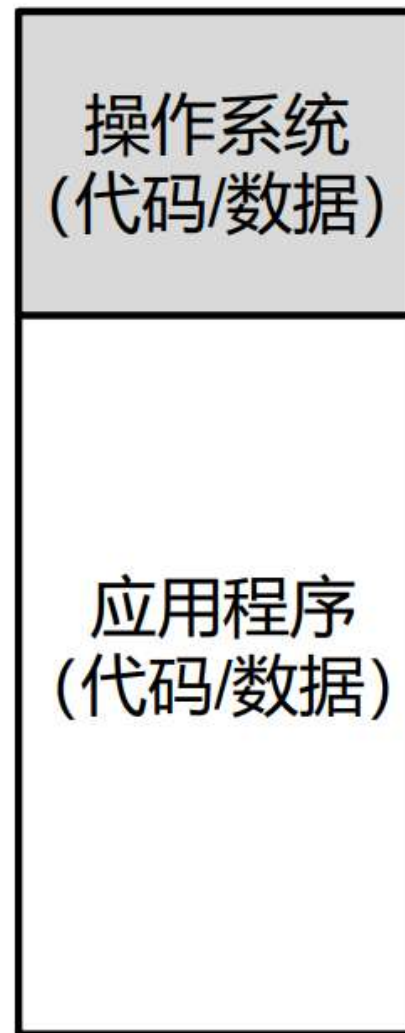
单道程序的内存管理

■ 硬件

- 物理内存容量小

■ 软件

- 单个应用程序 + （简单）操作系
- 直接面对物理内存编程
- 各自使用物理内存的一部分





单道程序的内存管理

条件:

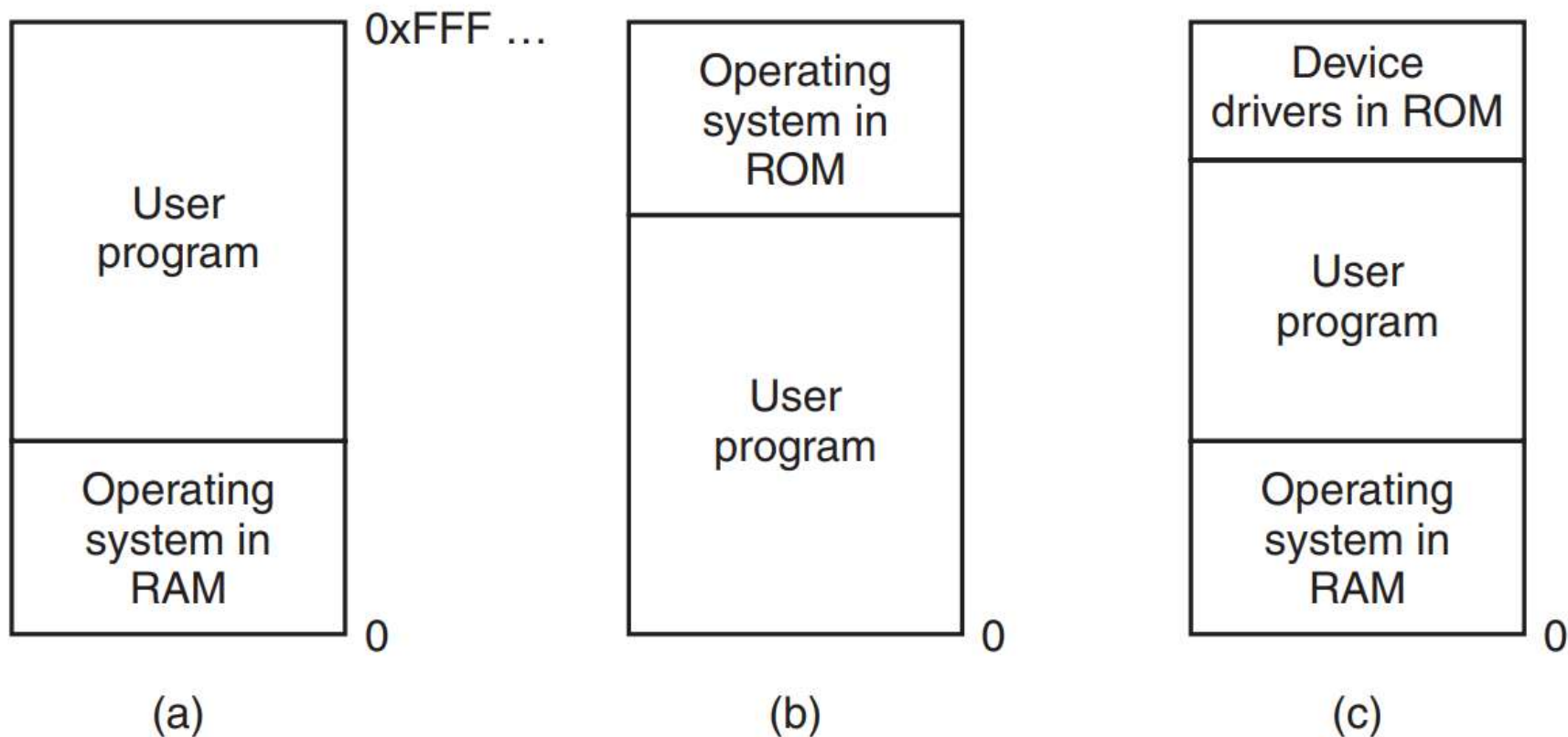
- 在单道程序环境下，整个内存里只有两个程序：一个用户程序和操作系统
- 操作系统所占的空间是固定的
- 因此可以将用户程序永远加载到同一个地址，即用户程序永远从同一个地方开始运行

结论:

- 用户程序的地址在运行之前可以计算



操作系统在物理内存中的位置





单道程序的内存管理

方法:

- **静态地址翻译**: 即在程序运行之前就计算出所有物理地址
- 静态翻译工作可以由加载器实现

分析:

- 地址独立? **YES**. 因为用户无需知道物理内存的相关知识
- 地址保护? **YES**. 因为没有其它用户程序



单道程序的内存管理

优点:

- 执行过程中无需任何地址翻译工作，程序运行速度快

缺点:

- 比物理内存大的程序无法加载，因而无法运行
- 造成资源浪费（小程序会造成空间浪费；I/O时间长会造成计算资源浪费）

思考:

- 程序可加载到内存中，就一定可以正常运行吗？
- 用户程序运行会影响操作系统吗？



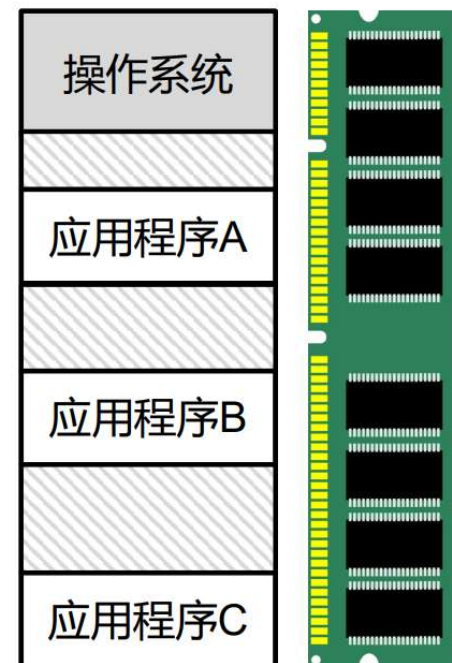
多道程序的存储分配

存储器分区 (Memory Partitioning)

- 把内存分为一些大小相等或不等的分区(partition), 每个应用程序占用一个或几个分区。操作系统占用其中一个分区
- 适用于多道程序系统和分时系统, 支持多个程序并发执行, 但难以进行内存分区的共享

方法:

- **固定 (静态) 式分区分配**, 程序适应分区
- **可变 (动态) 式分区分配**, 分区适应程序

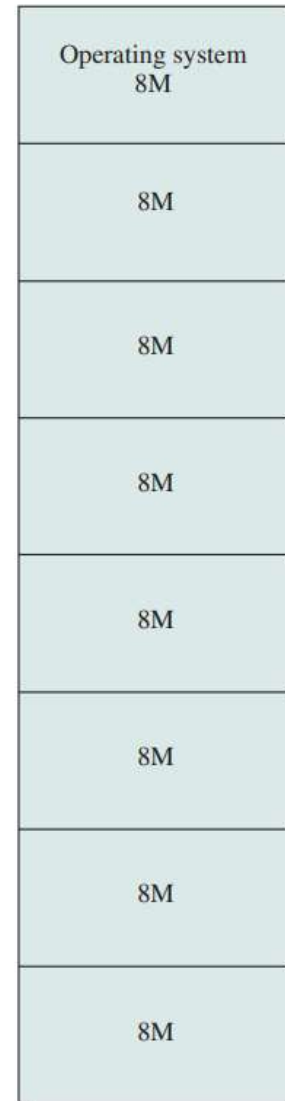




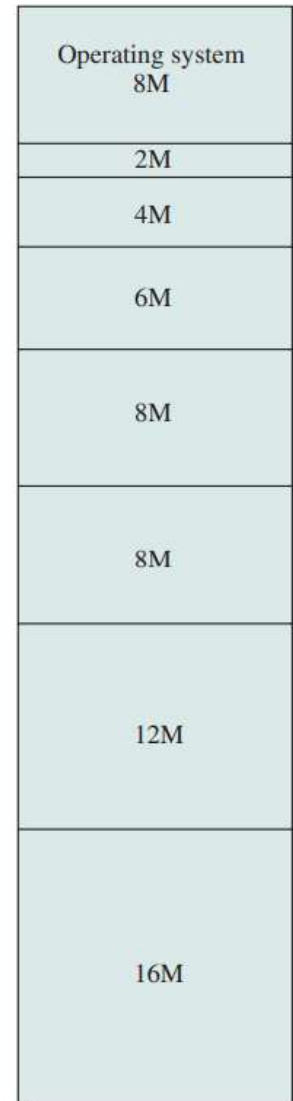
固定式分区 (Fixed Partitioning)

■ 把内存划分为若干个固定大小的连续分区

- **分区大小相等**：只适合于多个相同程序的并发执行（处理多个类型相同的对象）
- **分区大小不等**：多个小分区、适量的中等分区、少量的大分区。根据程序的大小，分配当前空闲的、适当大小的分区



(a) Equal-size partitions

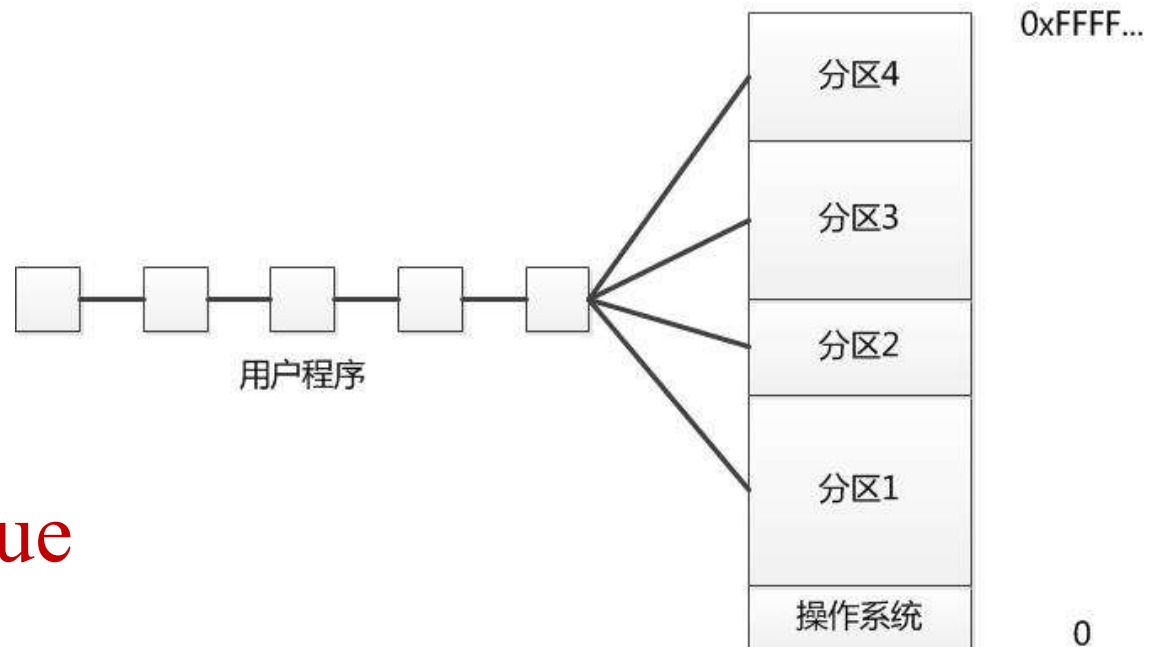


(b) Unequal-size partitions



单一队列的分配方式

- 当需要加载程序时，选择一个当前闲置且容量足够大的分区进行加载，可采用共享队列的固定分区（多个用户程序排在一个共同的队列里面等待分区）分配

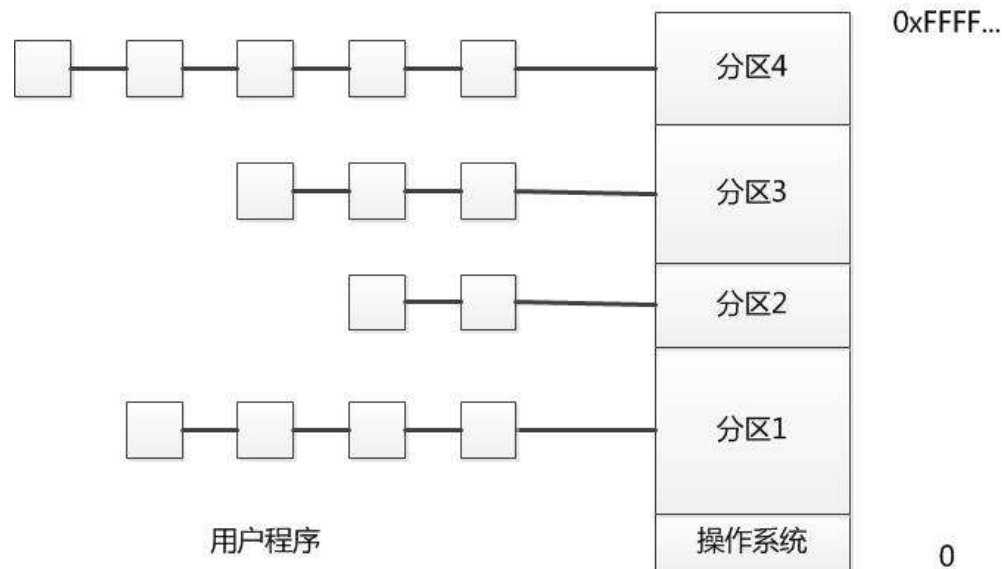


Single queue



多队列分配方式

- 由于程序大小和分区大小不一定匹配，有可能形成一个小程序占用一个大分区的情况，从而造成内存里虽然有小分区闲置但无法加载大程序的情况。这时，可以采用多个队列，给每个分区一个队列，程序按照大小排在相应的队列里



One process queue per partition



固定式分区的管理

■ 优点

- 易于实现，开销小

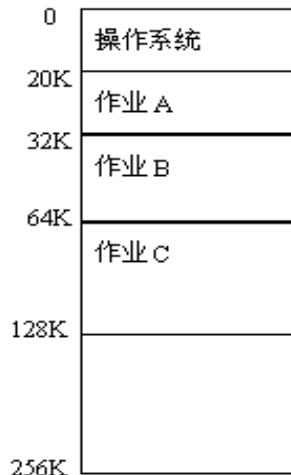
■ 缺点

- 内部碎片（internal fragmentation），造成空间浪费
- 分区总数固定，限制了并发执行的程序数目

■ 采用的数据结构：分区表记录分区的大小和使用情况

分区号	大小	起址	状态
1	12K	20K	已分配
2	32K	32K	已分配
3	64K	64K	已分配
4	128K	128K	未分配

分区说明表



存储空间分配情况



系统中的碎片

内存中无法被利用的存储空间称为碎片。

内部碎片（分配大小大于实际需要）：

- 指分配给作业的存储空间中未被利用的部分，如固定分区中存在的碎片
- 单一连续区存储管理、固定分区存储管理等都会出现内部碎片
- 内部碎片无法被整理，但作业完成后会得到释放。它们其实已经被分配出去了，只是没有被利用。





可变式分区 (Dynamic Partitioning)

■ 又称动态分区模式

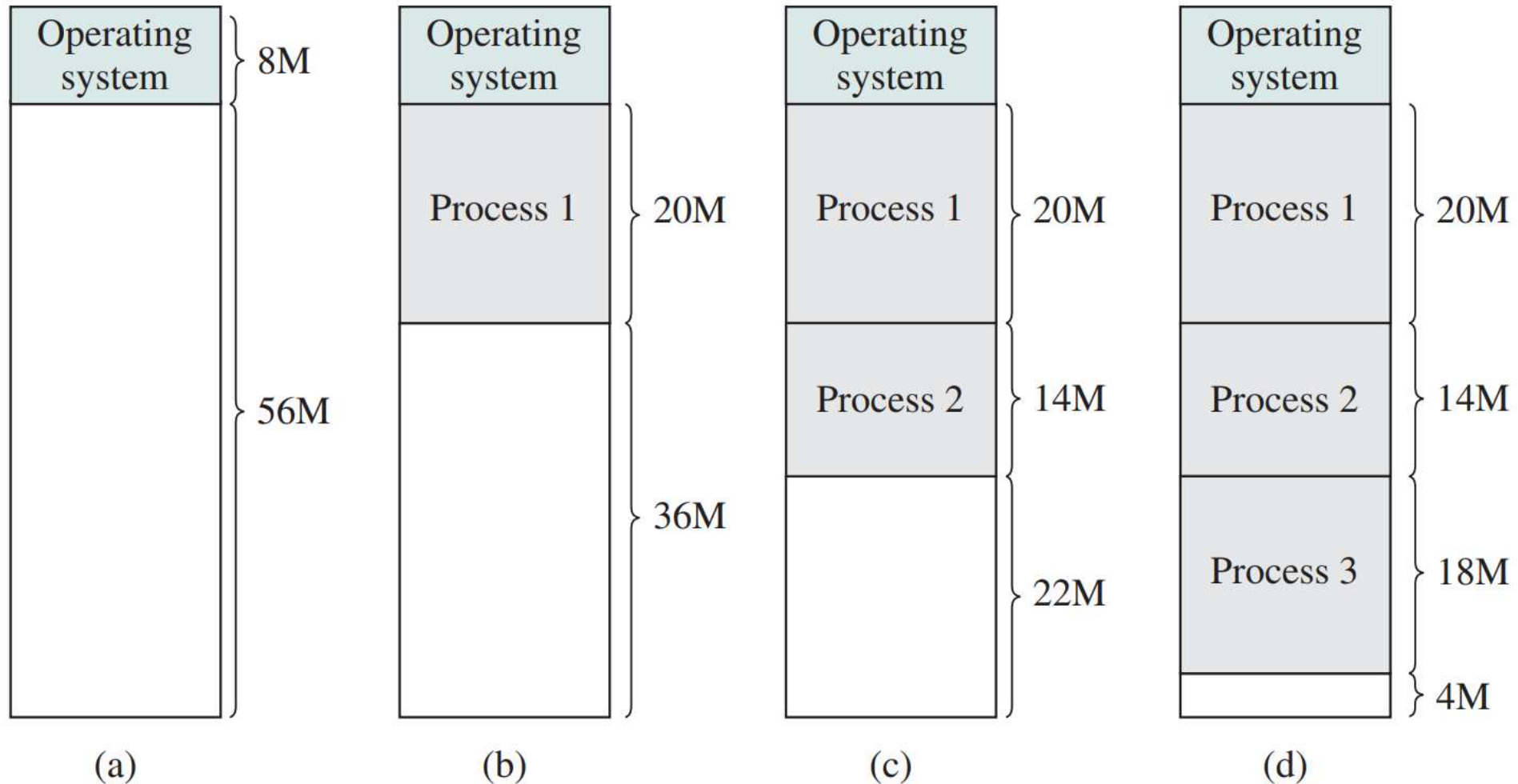
- 根据作业的实际需要，动态地为之分配内存空间
- 划分的时间、大小、位置都是动态的

■ 优点

- 没有内碎片
- 克服固定分区内存资源的浪费问题，有利于多道程序设计，提高内存资源利用率



可变式分区 (Dynamic Partitioning)





可变式分区

■ 动态分配数据结构

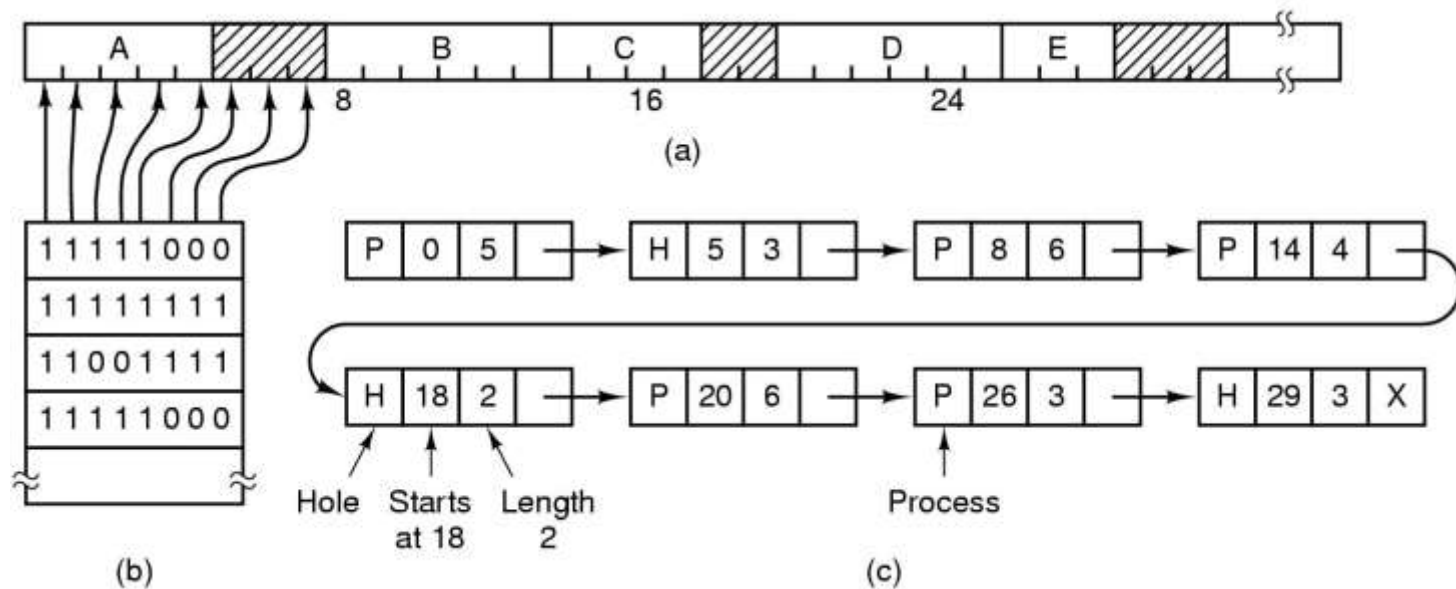
- 位图表示法
- 链表表示法

■ 动态分区分配算法

■ 动态分区的分配和回收

分配数据结构 —— 闲置空间的管理

- 在管理内存的时候，OS需要知道内存空间有多少空闲？这就必须跟踪内存的使用，跟踪的办法有两种：
位图表示法（分区表）和链表表示法（分区链表）





位图表示法

- 给每个分配单元赋予一个字位，用来记录该分配单元是否闲置。例如，字位取值为0表示单元闲置，取值为1则表示已被占用，这种表示方法就是位图表示法

1	1	1	1	1	1	0	0	0	1	0	1
---	---	---	---	---	---	---	---	---	---	---	---

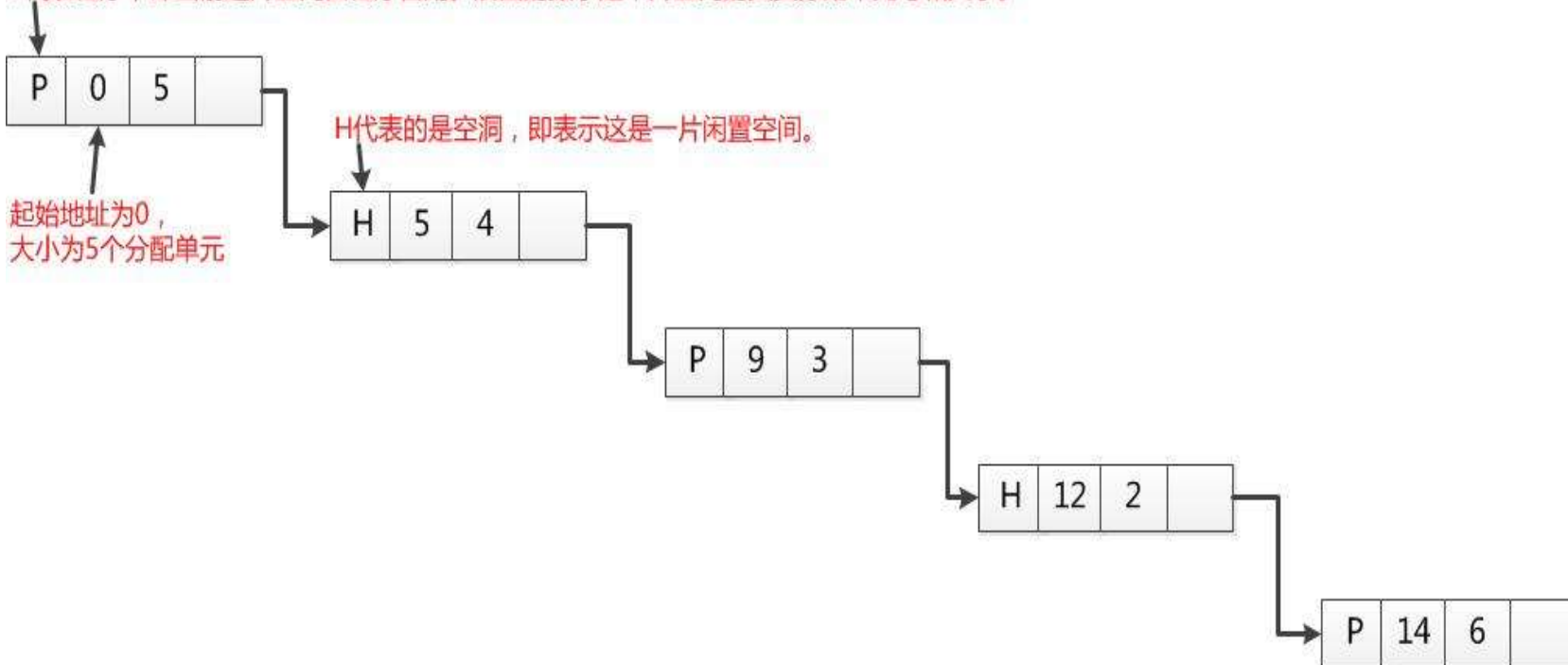
内存分配位图表示



链表表示法

- 将分配单元按照是否闲置链接起来，这种方法称为链表表示法。

P代表程序，即当前这片空间由程序占用。后面的数字是本片空间的其实分配单元号和大小。





两种方法的特点

■ 位图表示法:

- 空间成本固定：不依赖于内存中的程序数量
- 时间成本低：操作简单，直接修改其位图值即可
- 没有容错能力：如果一个分配单元为1，不能肯定应该为1还是因错误变成1

■ 链表表示法:

- 空间成本：取决于程序的数量
- 时间成本：链表扫描通常速度较慢，还要进行链表项的插入、删除和修改
- 有一定容错能力：因为链表有被占空间和闲置空间的表项，可以相互验证



动态分区分配算法 —— 顺序分配算法

基于顺序搜索的分配算法

■ 首次适应算法 (First Fit)

- 每个空白区按其在存储空间中地址递增的顺序连在一起，在为作业分配存储区域时，从这个空白区域链的始端开始查找，选择第一个足以满足请求的空白块

■ 下次适应算法 (Next Fit)

- 把存储空间中空白区构成一个循环链，每次为存储请求查找合适的分区时，总是从上次查找结束的地方开始，只要找到一个足够大的空白区，就将它划分后分配出去

■ 最佳适应算法 (Best Fit)

- 为一个作业选择分区时，总是寻找其大小最接近于作业所要求的存储区域

■ 最坏适应算法 (Worst Fit)

- 为作业选择存储区域时，总是寻找最大的空白区



算法举例

- 例：系统中的空闲分区表如下表示，现有三个作业分配申请内存空间 100K、30K 及 7K，给出按首次适应算法、下次适应算法、最佳适应算法和最坏适应算法的内存分配情况及分配后空闲分区表

区号	大小	起始地址	状态
1	32K	20K	未分配
2	8K	52K	未分配
3	120K	60K	未分配
4	331K	180K	未分配



首次适应算法 (First Fit)

- 申请作业 100k，分配 3 号分区，剩下分区为 20k，起始地址 160K
- 100K、30K、7K

区号	大小	起始地址	状态
1	32K	20K	未分配
2	8K	52K	未分配
3	20K	160K	未分配
4	331K	180K	未分配



首次适应算法 (First Fit)

- 申请作业 30K，分配 1 号分区，剩下分区为 2K，起始地址 50K
- 100K、30K、7K

区号	大小	起始地址	状态
1	2K	50K	未分配
2	8K	52K	未分配
3	20K	160K	未分配
4	331K	180K	未分配



首次适应算法 (First Fit)

- 申请作业 7K，分配 2 号分区，剩下分区为 1K，起始地址59K。
- 100K、30K、7K

区号	大小	起始地址	状态
1	2K	50K	未分配
2	1K	59K	未分配
3	20K	160K	未分配
4	331K	180K	未分配

优先利用内存低地址部分的空闲分区。但由于低地址部分不断被划分，留下许多难以利用的很小的空闲分区（碎片或零头），而每次查找又都是从低地址部分开始，增加了查找可用空闲分区的开销



下次适应算法 (Next Fit)

- 按下次适应算法，申请作业 100k，分配 3 号分区，剩下分区为20k，起始地址 160K；
- 100K、30K、7K

区号	大小	起始地址	状态
1	32K	20K	未分配
2	8K	52K	未分配
3	20K	160K	未分配
4	331K	180K	未分配



下次适应算法 (Next Fit)

- 按下次适应算法，申请作业 30K，分配 4 号分区，剩下分区为301K；
- 100K、30K、7K

区号	大小	起始地址	状态
1	32K	20K	未分配
2	8K	52K	未分配
3	20K	160K	未分配
4	301K	210K	未分配



下次适应算法 (Next Fit)

- 按下次适应算法，申请作业 7k，分配 4 号分区
- 100K、30K、7K

区号	大小	起始地址	状态
1	32K	20K	未分配
2	8K	52K	未分配
3	20K	160K	未分配
4	294K	217K	未分配

使存储空间的利用更加均衡，不致使小的空闲区集中在存储区的一端，但这会导致缺乏大的空闲分区



最佳适应算法

■ 分配顺序：100K、30K、7K

按容量大小递增的次序排列

分配前的空闲分区表

区号	大小	起址
1	8k	52k
2	32k	20k
3	120k	60k
4	331k	180k

作业30K分配后

区号	大小	起址
2	2k	50k
1	8k	52k
3	20k	160k
4	331k	180k

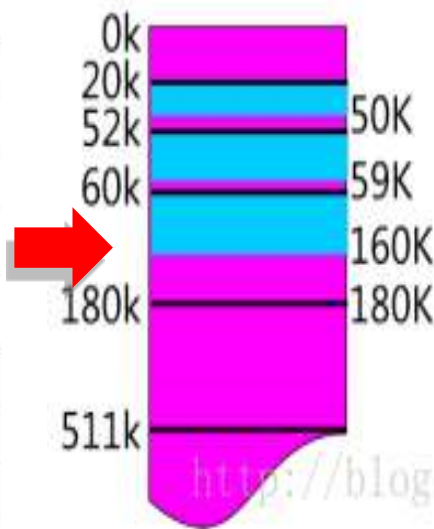
按容量递增的次序重新排列

作业100K分配后

区号	大小	起址
1	8k	52k
3	20k	160k
2	32k	20k
4	331k	180k

作业7K分配后

区号	大小	起址
1	1k	59k
2	2k	50k
3	20k	160k
4	331k	180k



区号	大小	起址
1	1k	59k
2	2k	50k
3	20k	160k
4	331k	180k



最坏适应算法

分配顺序：100K、30K、7K

按容量大小递减的次序排列

分配前的空闲分区表

区号	大小	起址
1	331k	180k
2	120k	60k
3	32k	20k
4	8k	52k

作业30K分配后

区号	大小	起址
1	201k	310k
2	120k	60k
3	32k	20k
4	8k	52k

按容量递减的次序重新排列

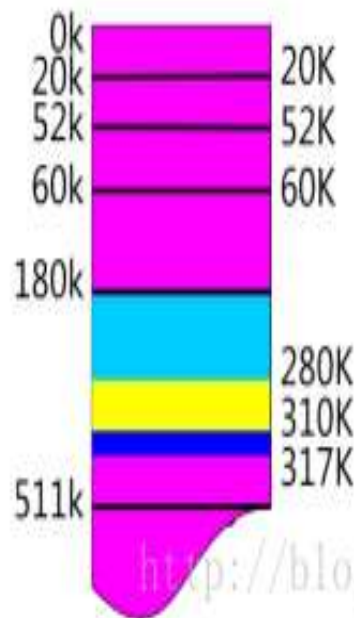
作业100K分配后

区号	大小	起址
1	231k	280k
2	120k	60k
3	32k	20k
4	8k	52k

作业7K分配后

区号	大小	起址
1	194k	317k
2	120k	60k
3	32k	20k
4	8k	52k

大作业存储空间的申请
往往会得不到满足



区号	大小	起址
1	194k	317k
2	120k	60k
3	32k	20k
4	8k	52k



算法特点

- **首次适应：** 优先利用内存低地址部分的空闲分区。但由于低地址部分不断被划分，留下许多难以利用的很小的空闲分区（碎片或零头），而每次查找又都是从低地址部分开始，增加了查找可用空闲分区的开销
- **下次适应：** 使存储空间的利用更加均衡，不致使小的空闲区集中在存储区的一端，但这会导致缺乏大的空闲分区



算法特点

- **最佳适应算法：**若存在与作业大小一致的空闲分区,则它必然被选中，若不存在与作业大小一致的空闲分区，则只划分比作业稍大的空闲分区，从而保留了大的空闲分区。最佳适应算法往往使剩下的空闲区非常小，**从而在存储器中留下许多难以利用的小空闲区（碎片）**
- **最坏适应算法：**总是挑选满足作业要求的最大的分区分配给作业。这样使分给作业后剩下的空闲分区也较大，可装下其它作业。由于最大的空闲分区总是因首先分配而划分，当有大作业到来时，其存储空间的申请往往会得不到满足



练习题

- 在下列存储管理算法中，内存分配和释放平均时间之和为最大的是：

A 首次适应 B 下次适应 C 最佳适应 D 最差适应



练习题

- 在下列存储管理算法中，内存分配和释放平均时间之和为最大的是：

A 首次适应 B 下次适应 C 最佳适应 D 最差适应

需要寻找满足需要且最小的空闲块，
释放要找到上下邻空闲区，修改插入链表



练习题

■ 可变分区又称为动态分区，它是在系统运行过程中____动态建立的：

A 作业未装入

B 在作业装入

C 在作业创建

D 在作业完成



练习题

- 可变分区又称为动态分区，它是在系统运行过程中____动态建立的：

A 作业未装入

B 在作业装入

C 在作业创建

D 在作业完成

分区大小在程序装入时大小动态确定，量身定制



练习：可变分区的内存分配

- 假设一个可变分区系统中内存按照顺序包含如下空闲区：10 KB, 4 KB, 20 KB, 18 KB, 7 KB, 9 KB, 12 KB, and 15 KB.
- 假设后续连续内存分配请求是：(a) 12 KB (b) 10 KB (c) 9 KB
- 对于首次适应那个空闲区会被分配？对于最佳适应, 最差适应, 和下次适应又会分配那些空闲区？



练习：可变分区的内存分配

- 首次适应 20 KB, 10 KB, 18 KB.
- 最佳适应 12 KB, 10 KB, 9 KB.
- 最差适应 20 KB, 18 KB, 15 KB.
- 下次适应 20 KB, 18 KB, 9 KB.



基于索引搜索的分配算法

- 基于顺序搜索的动态分区分配算法一般只是适合于较小的系统，如果系统的分区很多，空闲分区表（链）可能很大（很长），检索速度会比较慢。为了提高搜索空闲分区的速度，大中型系统采用了基于索引搜索的动态分区分配算法。

```
// 顺序搜索:  $O(n)$   
for (free_block = free_list; free_block != NULL; free_block = free_block->next) {  
    if (free_block->size >= request_size) {  
        // 找到合适的块  
        break;  
    }  
}  
  
// 当  $n = 10,000$  时，每次分配都可能遍历数千个节点
```



快速适应算法

- **快速适应算法**，又称为分类搜索法，把空闲分区按容量大小进行分类，经常用到长度的空闲区设立单独的空闲区链表。系统为多个空闲链表设立一张管理索引表。

- **优点：**

- 查找效率高，仅需要根据程序的长度，寻找到能容纳它的最小空闲区链表，取下第一块进行分配即可。该算法在分配时，不会对任何分区产生分割，所以能保留大的分区，也不会产生内存碎片。

- **缺点：**

- 在分区归还主存时算法复杂，系统开销大。在分配空闲分区时是以进程为单位，一个分区只属于一个进程，存在一定的浪费。空间换时间。



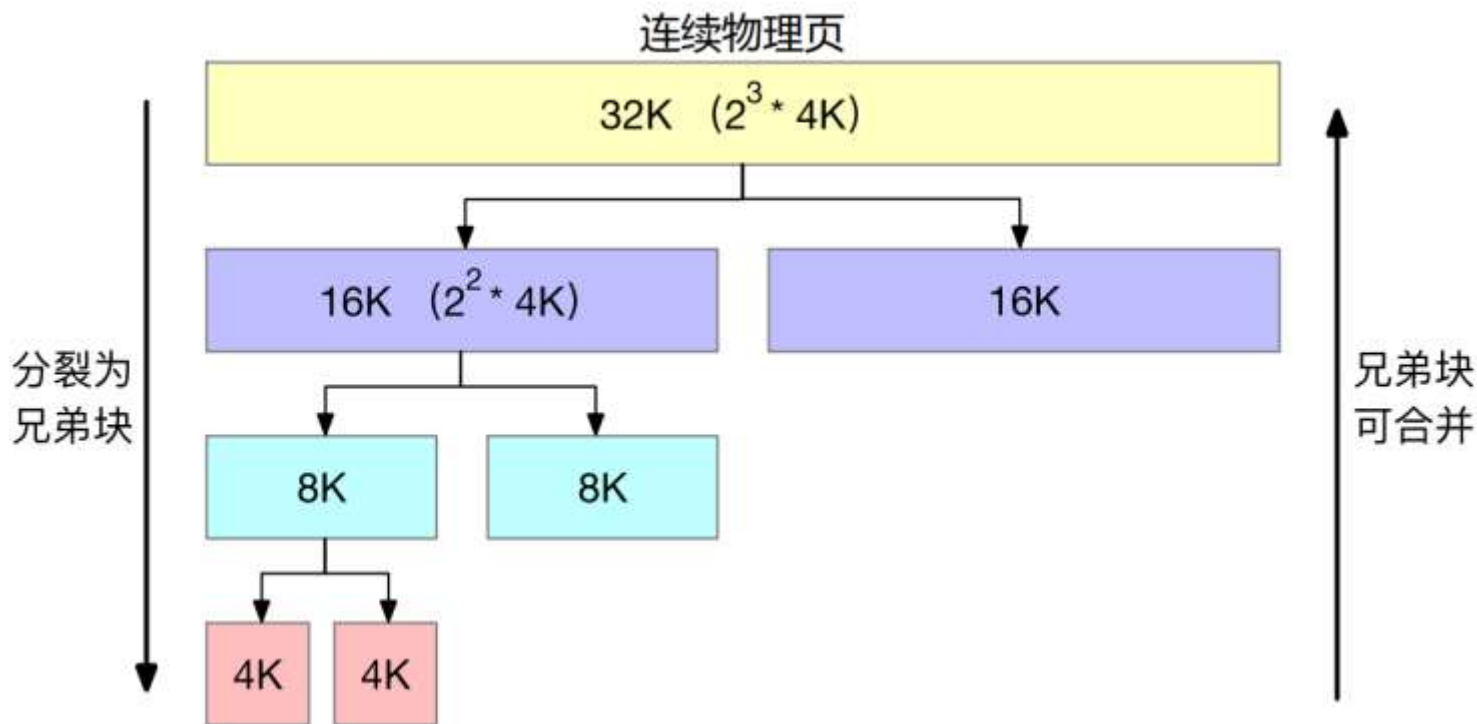
伙伴系统 (Buddy System)

- 固定分区方式不够灵活，当进程大小与空闲分区大小不匹配时，内存空间利用率很低
- 动态分区方式算法复杂，回收空闲分区时需要进行分区合并等，系统开销较大
- 伙伴系统 (buddy system)是介于固定分区与可变分区之间的动态分区技术
 - 伙伴：在分配存储块时将一个大的存储块分裂成两个大小相等的小块，这两个小块就称为“伙伴”

伙伴系统 (Buddy System)

■ 伙伴系统:

- 伙伴系统是一种动态内存分配算法，它将内存块分割成大小为2的幂次方的块
- 这些块被称为“伙伴”，因为它们总是成对出现（大小相同）





伙伴系统 (Buddy System)

■ 核心思想:

- 将内存划分为大小为 2^k 的块
- 每个内存块都有一个“伙伴” (Buddy)，即与其大小相等且地址相邻的块

■ 分配规则:

- 根据请求大小，找到最接近且大于等于的2的幂次块
- 如果块太大，则分裂为两个相等的“伙伴”

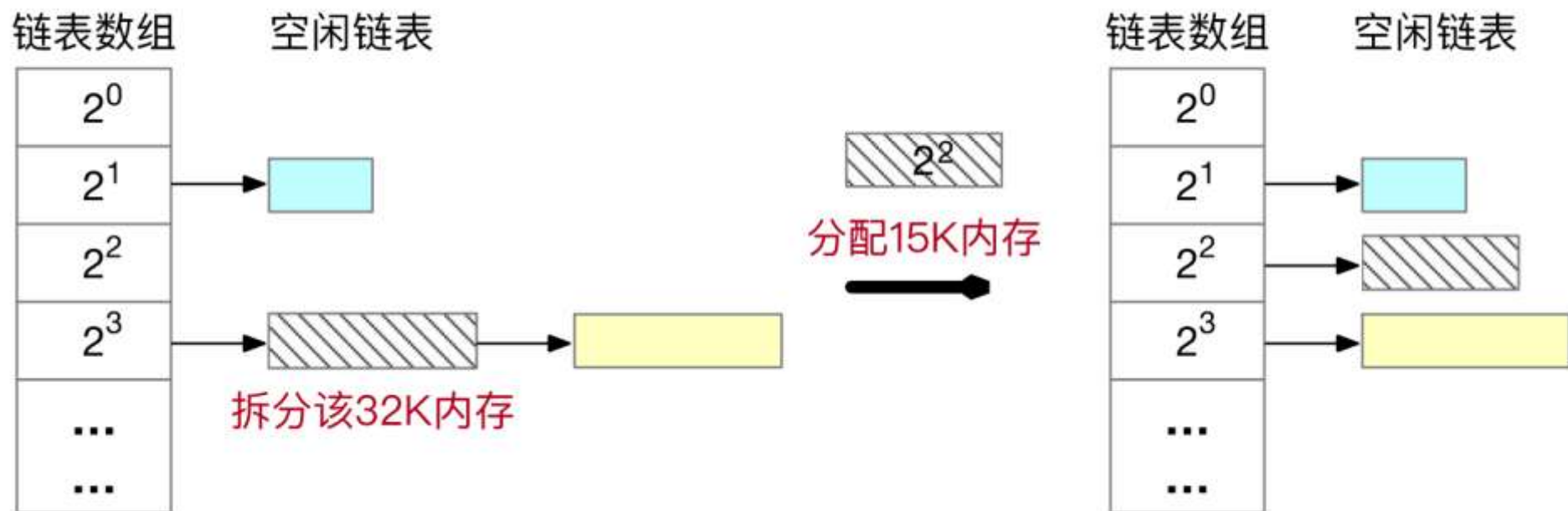
■ 回收规则:

- 释放内存时，检查其伙伴是否空闲，若空闲则合并成更大的块



伙伴系统实现

■ 分配合适大小的块：什么是“合适”？



■ 思考：分裂和合并都是级联操作，什么时候会级联？



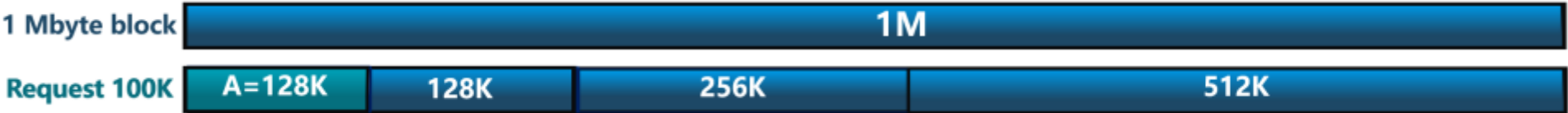
伙伴系统例子

1 Mbyte block

1M



伙伴系统例子





伙伴系统例子





伙伴系统例子

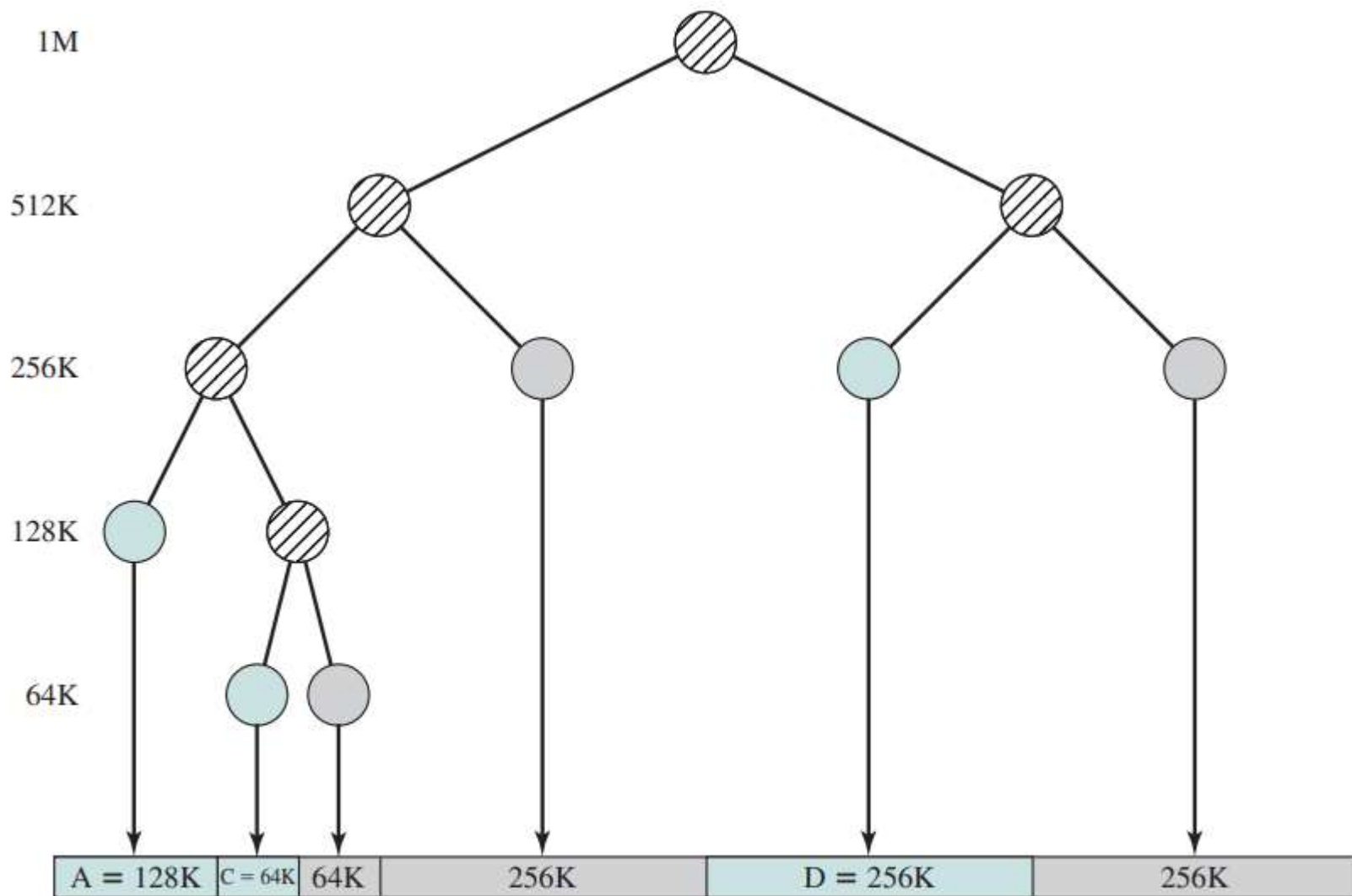




伙伴系统例子



伙伴系统的树状表示方式



Leaf node for
allocated block



Leaf node for
unallocated block



Non-leaf node



物理内存管理的评价指标

■ 内存资源利用率

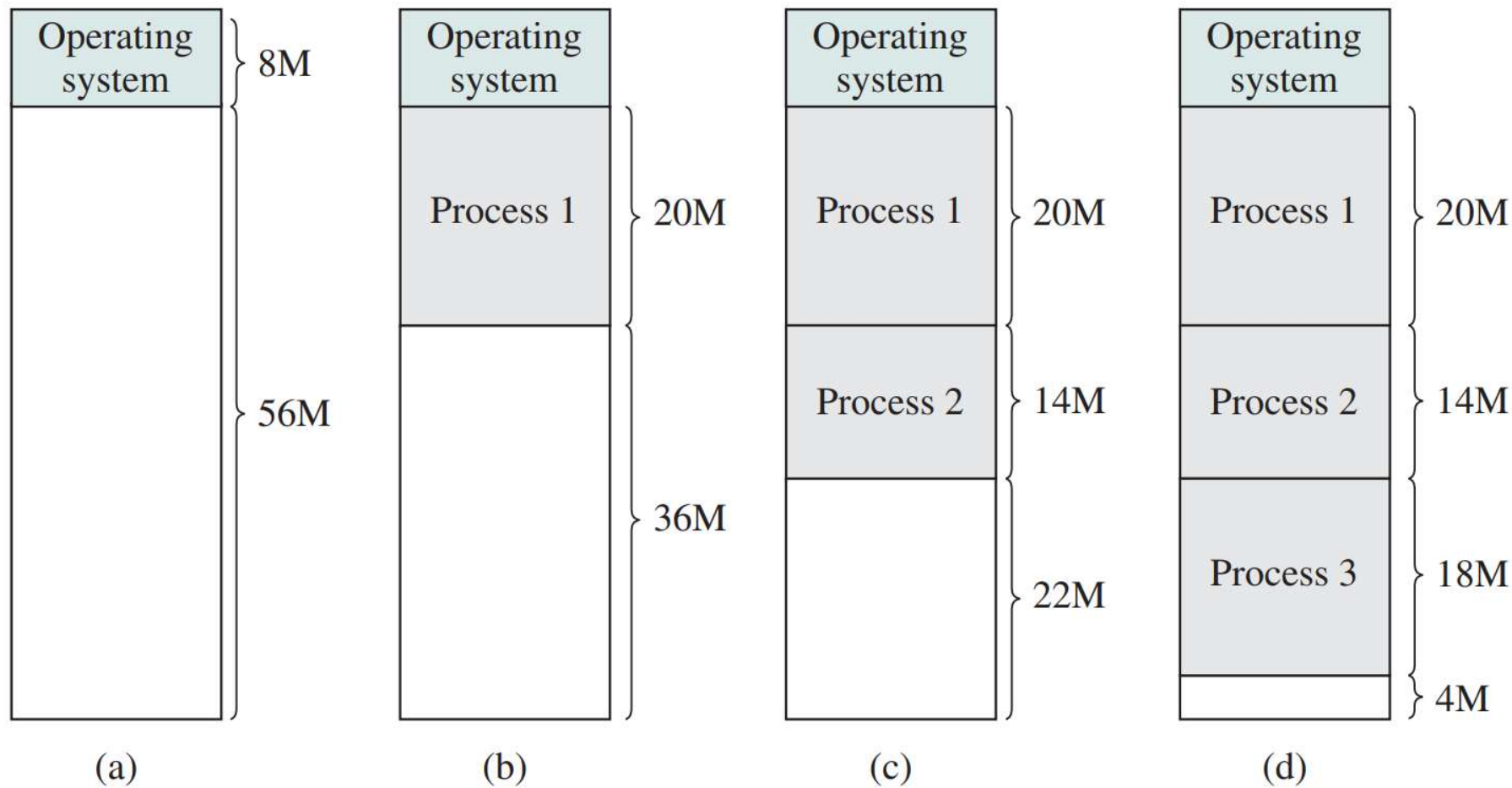
- 外部碎片和内部碎片

■ 分配速度

- 复杂的算法可以更好地解决碎片问题
- 但是内存分配操作的性能同样重要

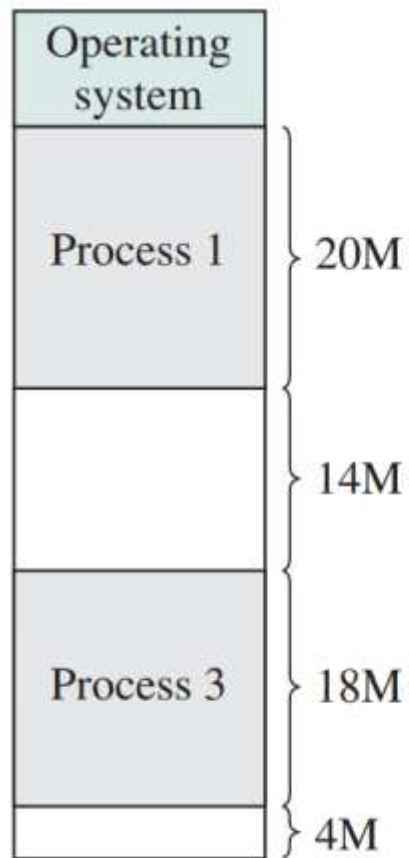


可变式分区产生外部碎片

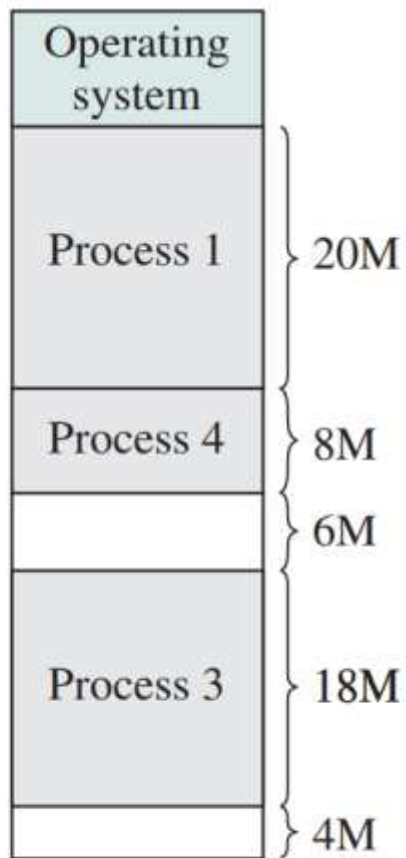




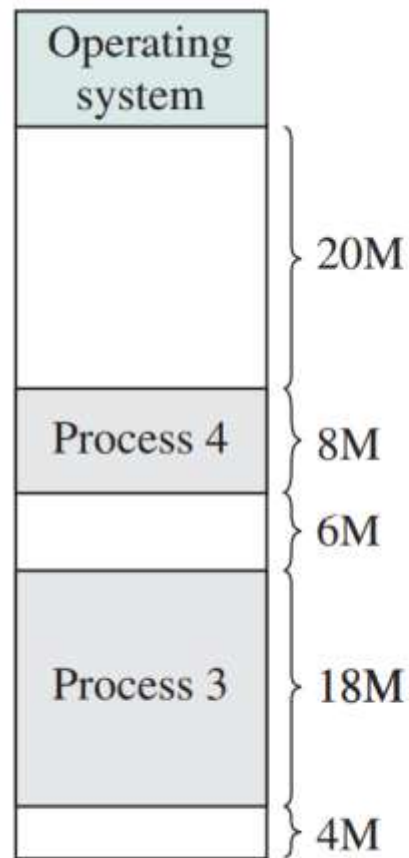
可变式分区产生外部碎片



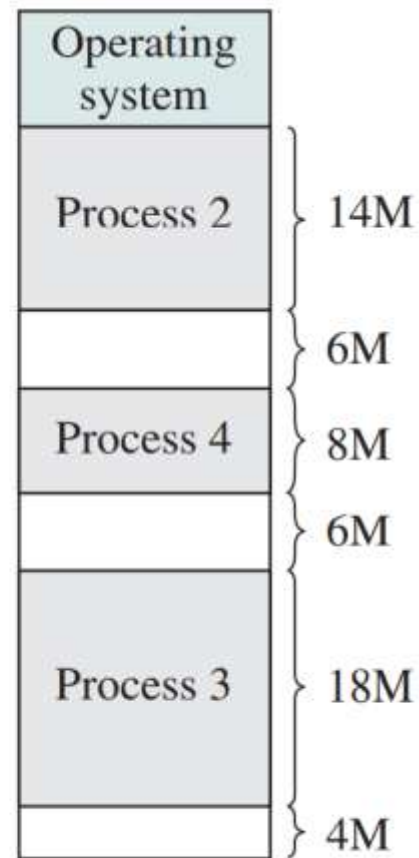
(e)



(f)



(g)



(h)



系统中的碎片

外部碎片（空闲的但不连续，无法被使用）

- 指系统中无法利用的小的空闲分区。如分区与分区之间存在的碎片。这些不连续的区间就是外部碎片。动态分区管理会产生外部碎片。
- **外部碎片才是造成内存系统性能下降的主要原因**
- 如何消除外部碎片？





系统中的碎片

■ 内部碎片：分配大小大于实际使用



■ 外部碎片：空闲的但不连续，无法被使用

➤ 即使空闲部分加起来大于请求分配的大小，仍无法被使用



■ 如何消除内部碎片和外部碎片是内存管理的一个主要问题

➤ 伙伴系统、紧凑技术、段/页式内存管理



紧凑技术 (Compaction)

- 通过移动作业从把多个分散的小分区拼接成一个大分区的方法称为紧凑（拼接或紧缩）。
- 目标：消除外部碎片，使本来分散的多个小空闲分区连成一个大的空闲区。
- 紧凑时机：找不到足够大的空闲分区且总空闲分区容量可以满足作业要求时。

实现支撑

动态重定位：作业在内存中的位置发生了变化，这就必须对其地址加以修改或变换。

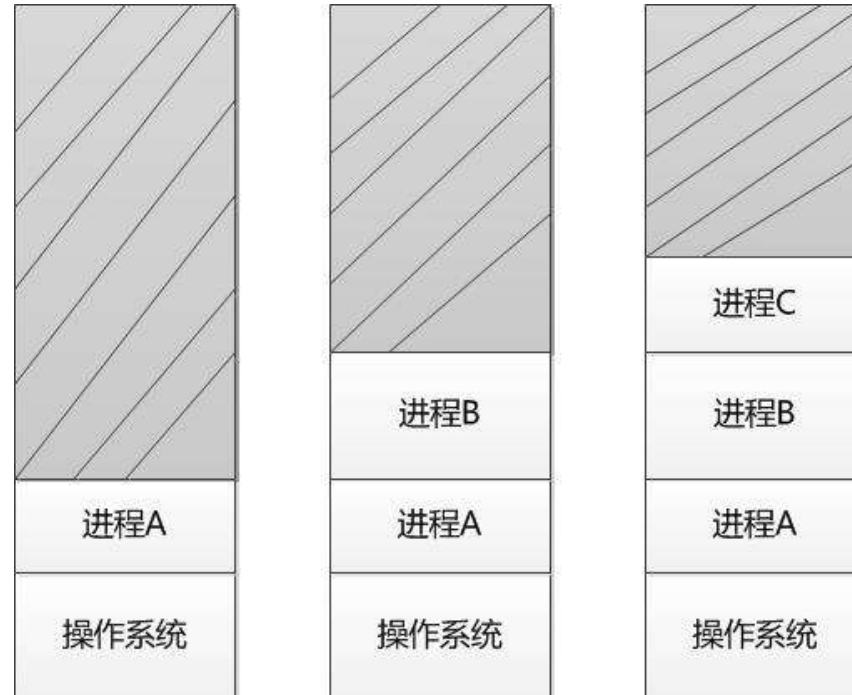
操作系统
作业A
20KB
作业B
30KB
作业C
15KB
作业D
25KB

操作系统
作业A
作业B
作业C
作业D
20KB
30KB
15KB
25KB



内存扩充的引入

- 例如，一开始内存中只有OS，这时候进程A来了，于是分出一片与进程A大小一样的内存空间；随后，进程B来了，于是在进程A之上分出一片给进程B；然后进程C来了，就在进程B上面再分出一片给C。

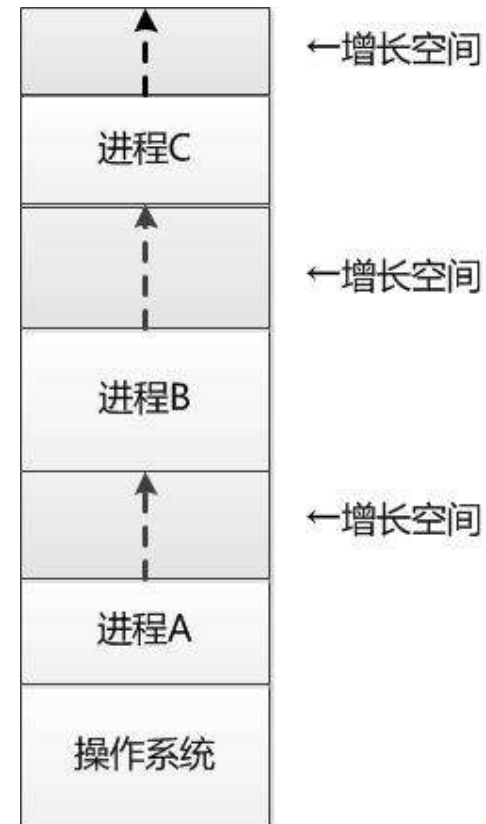




内存扩充的引入

- 每个程序像叠罗汉一样累计，如果程序B在成型过程中需要更多空间怎么办？（例如在实际程序中，很多递归嵌套函数调用的时候会回造成栈空间的增长）
- 预留一定的空间？OS怎么知道应该分配多少空间给一个程序呢？分配多了，就是浪费；而分配少了，则可能造成程序无法继续执行。

必须解决大作业在小内存中运行的问题。





解决的方法 — 内存扩充

- **覆盖与交换**技术是在多道程序环境下用来扩充内存的两种方法
- 覆盖与交换可以解决在小的内存空间运行大作业的问题，是“扩充”内存容量和提高内存利用率的有效措施
- 覆盖技术主要用在早期的OS 中，交换技术则用在现代OS 中

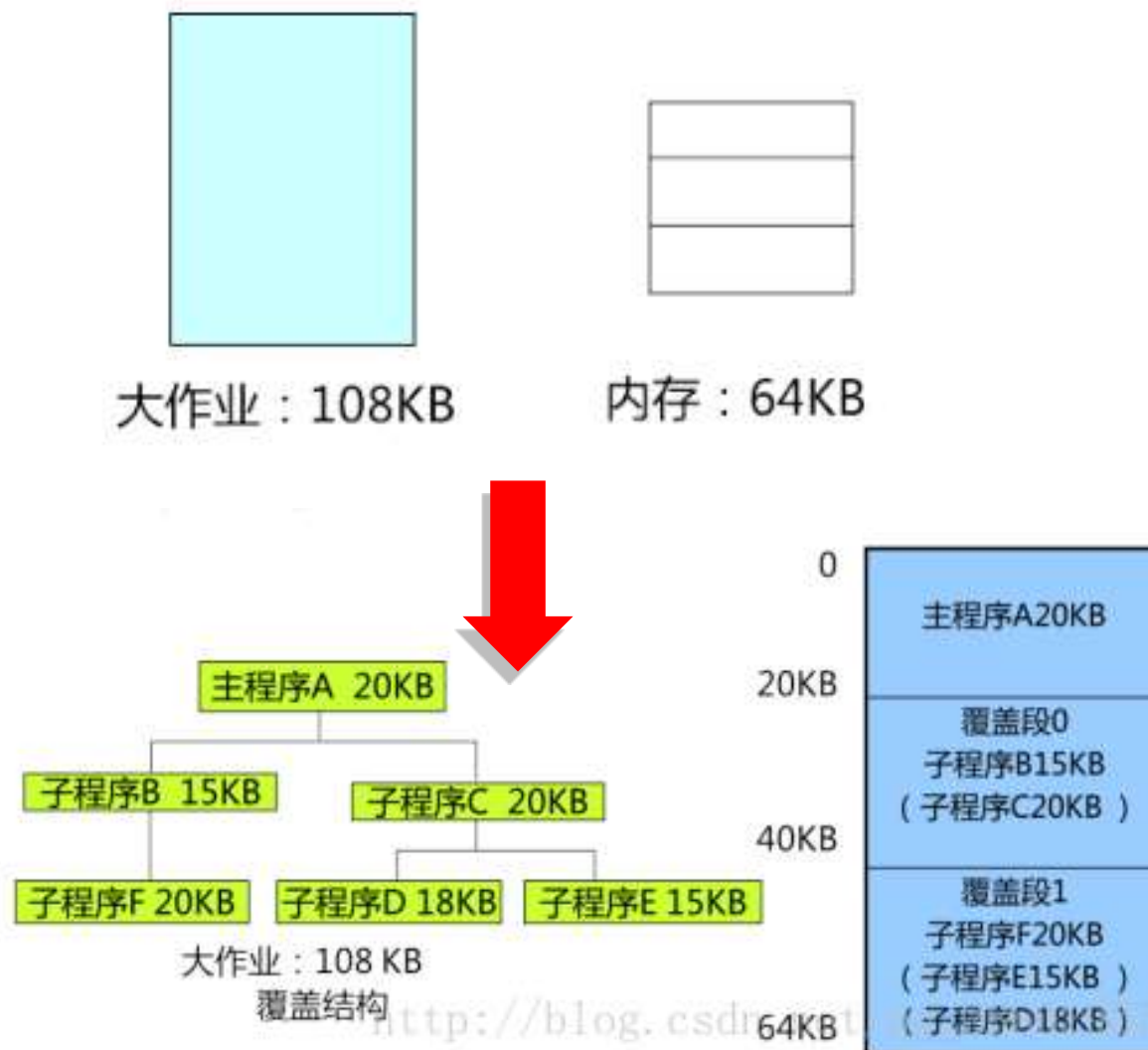


覆盖 (Overlay)

- 覆盖技术主要用在早期的 OS 中（内存 <64KB），可用的存储空间受限，某些大作业不能一次全部装入内存，产生了大作业与小内存的矛盾
- 覆盖：把一个程序划分为一系列功能相对独立的程序段，让执行时不求同时装入内存的程序段组成一组（称为覆盖段），共享主存的同一个区域，这种内存扩充技术就是覆盖
- 程序段先保存在磁盘上，当有关程序段的前一部分执行结束，把后续程序段调入内存，覆盖前面的程序段（内存“扩大”了）
- 一般要求作业各模块之间有明确的调用结构，程序员要向系统指明覆盖结构，然后由操作系统完成自动覆盖
- 缺点：对用户不透明，增加了用户负担



覆盖示例



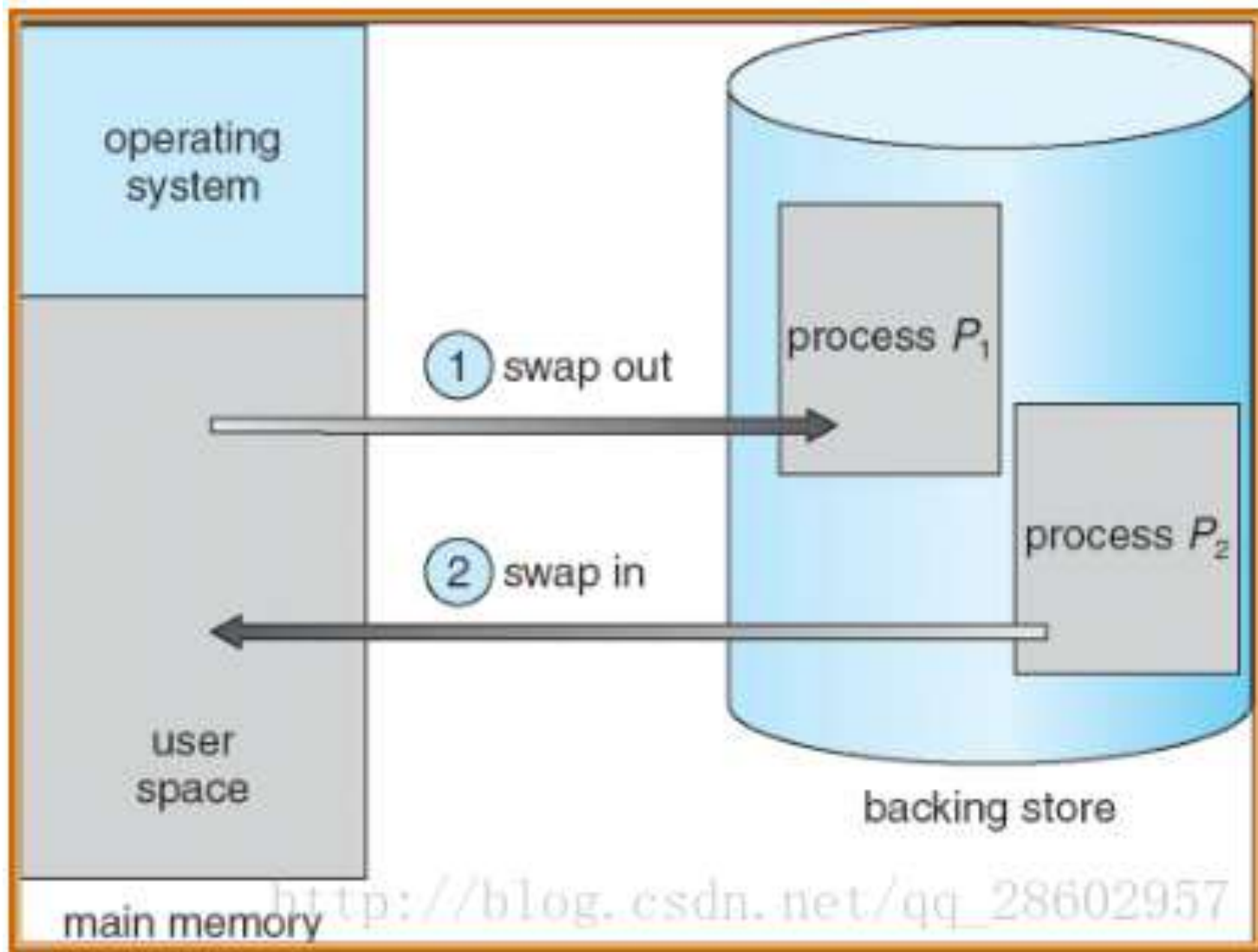


交换 (Swapping)

- **交换：**广义的说，所谓交换就是把暂时不用的某个（或某些）程序及其数据的部分或全部从主存移到辅存中去，以便腾出必要的存储空间；接着把指定程序或数据从辅存读到相应的主存中，并将控制转给它，让其在系统上运行。
- **优点：**增加并发运行的程序数目，并且给用户提供适当的响应时间；编写程序时不影响程序结构
- **缺点：**对换入和换出的控制增加处理机开销；程序整个地址空间都进行传送，没有考虑执行过程中地址访问的统计特性



交换示例





交换技术的几个问题

■ 选择原则，即将哪个进程换出/内存？

- 等待I/O的进程

■ 交换时机的确定，何时需发生交换？

- 只要不用就换出（很少再用）；只在内存空间不够或有不够的危险时换出

■ 交换时需要做哪些工作？

■ 换入回内存时位置的确定



覆盖与交换技术的区别

- **覆盖可减少一个程序运行所需的空間。交换可让整个程序暂存于外存中，让出内存空间**
- **覆盖是由程序员实现的，操作系统根据程序员提供的覆盖结构来完成程序段之间的覆盖。交换技术不要求程序员给出程序段之间的覆盖结构**
- **覆盖技术主要对同一个作业或程序进行。交换换主要在作业或程序间之间进行**